

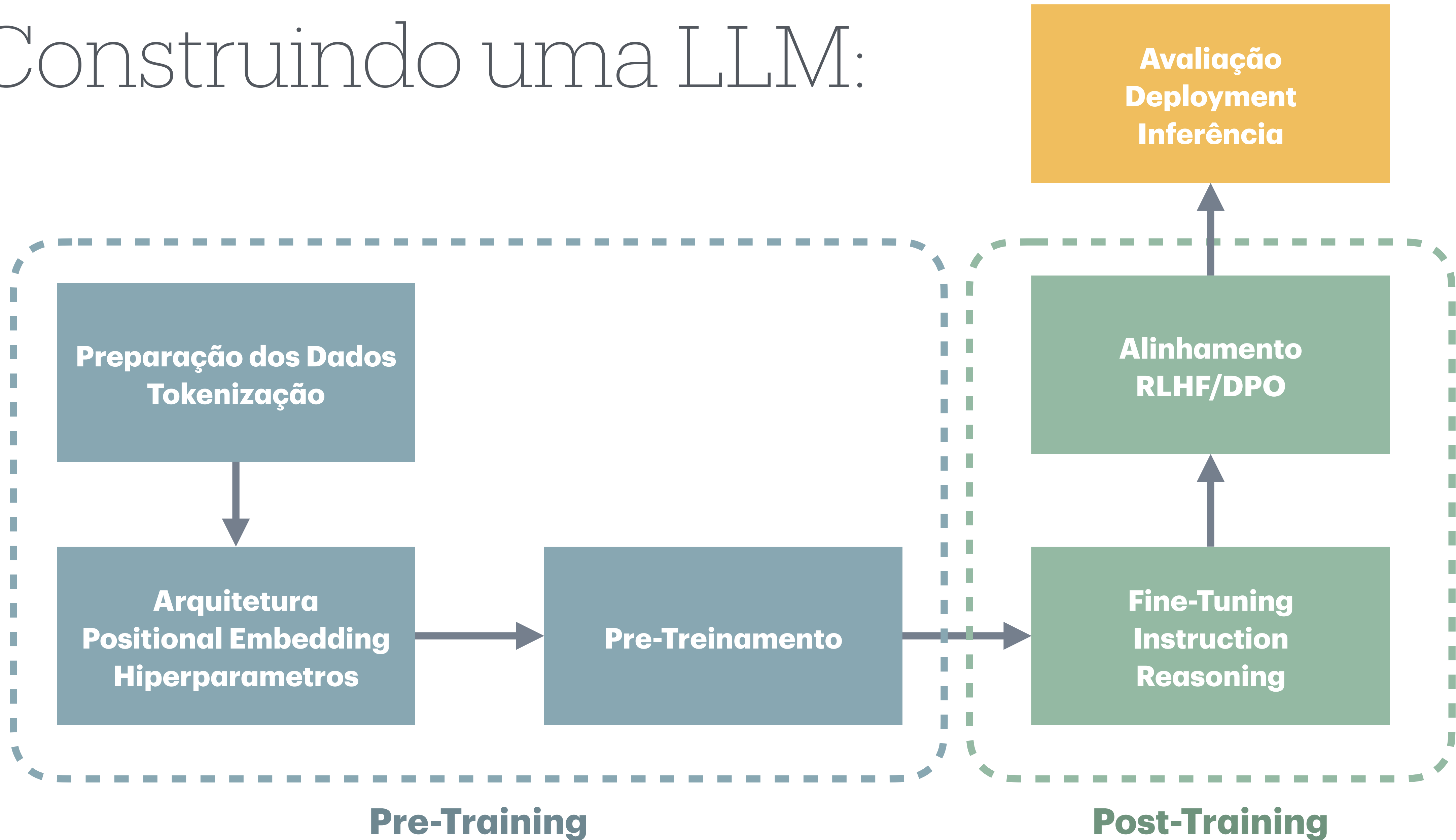
Large Language Models

Aula 17 - Machine Learning II

Daniel Yukimura, Francisco Ganacim
IMPA

3 de junho de 2026

Construindo uma LLM:

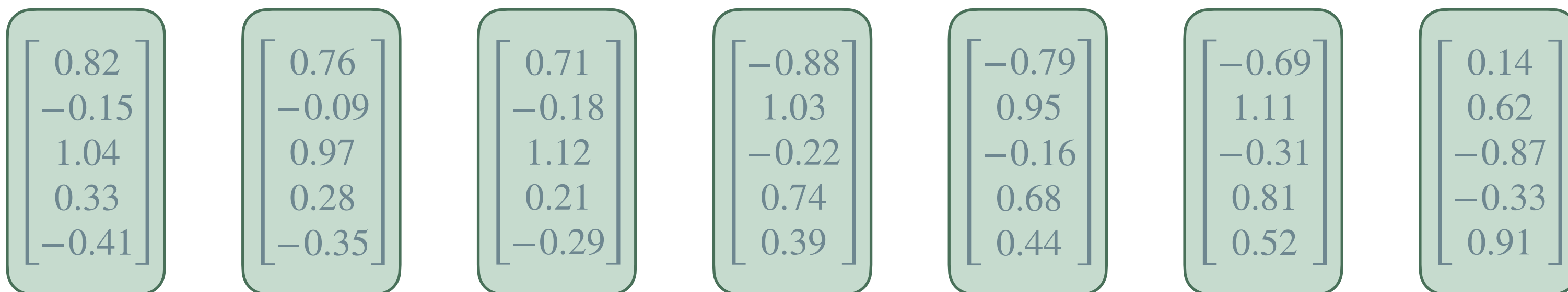


Tokenização

Essa manhã eu levei meu cachorro para



Essa manhã eu levei meu cachorro para



Tokenização

Palavras:

- Ex: jogo, joga, jogar, jogam, jogou, joga -> 6 tokens
- $|\mathcal{V}| \sim 50k$ para uma língua
- $|\mathcal{V}| > 100M$ para Web
- [UNK] -> comum

Caracteres:

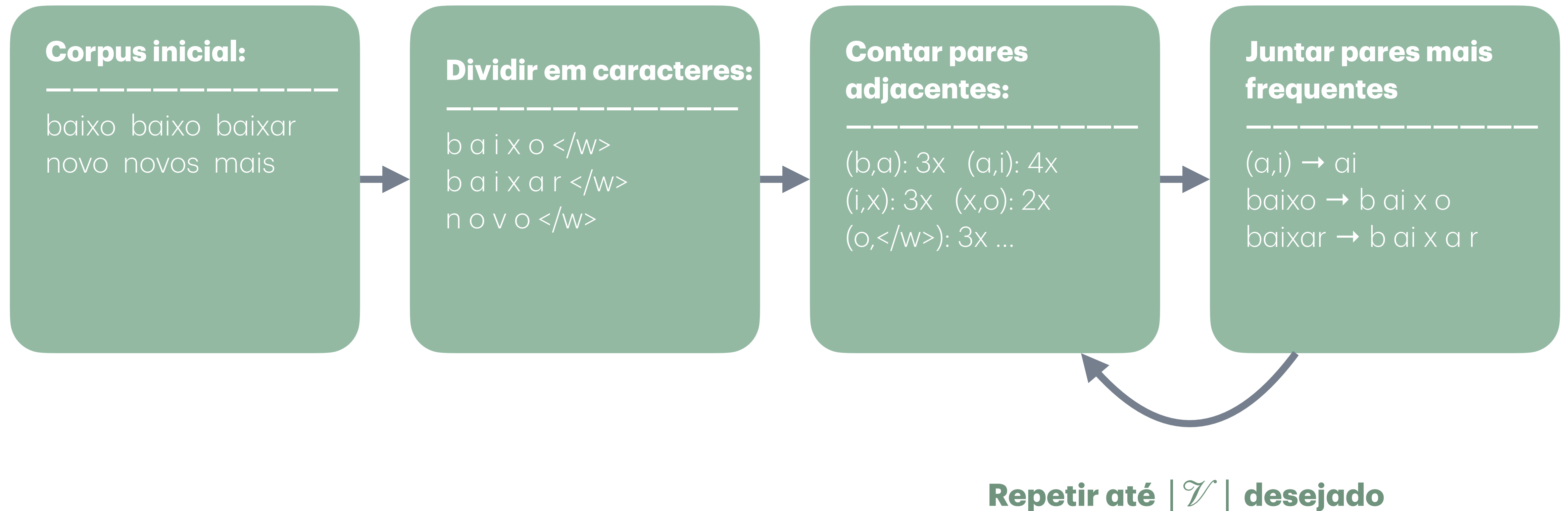
- Ex: Jogador -> 7 tokens
- $|\mathcal{V}| \sim 100-300$
- Sentenças são representadas por **sequências muito longas**

Subpalavras:

- Ex: jogador -> "jog"+"ador"
- Vocabulário compacto
- Sentenças com tamanho razoável
- [UNK] -> raro ou inexistente

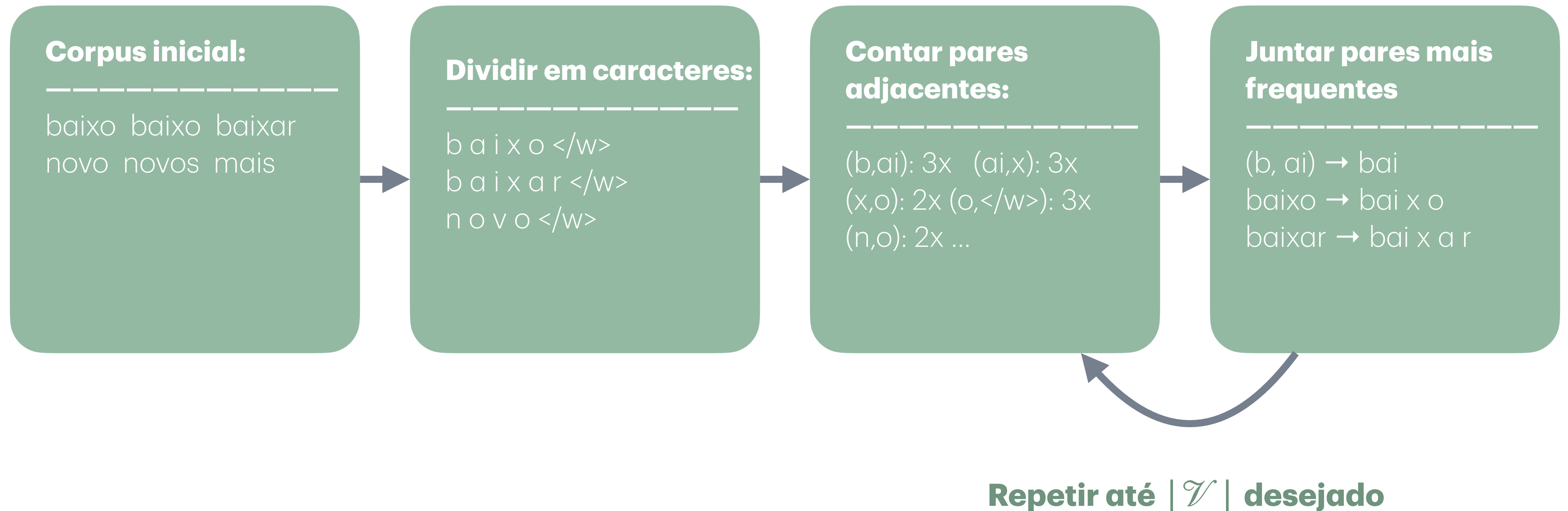
Tokenização

Algoritmo Byte-Pair Encoding (BPE)



Tokenização

Algoritmo Byte-Pair Encoding (BPE)



Tokenização

Algoritmo Byte-Pair Encoding (BPE)

Exemplos:

- jogadores -> jog + ador + es
- computador -> comput + ador
- brincando -> brinc + ando
- falando -> fal + ando
- tokenização -> token + ização

Tokenização

Algoritmo Byte-Pair Encoding (BPE)

Operações por byte:

- 1 byte = 8 bits — 256 valores possíveis
- UTF-8 codifica qualquer caractere do como sequência de 1–4 bytes:
 - A → 1 byte (65)
 - é → 2 bytes (195, 169)
 - 中 → 3 bytes (228, 184, 173)
 - 😊 → 4 bytes (240, 159, 152, 138)

Tokenização

Algoritmo Byte-Pair Encoding (BPE)

- **Treino :** Aprende regras para juntar bytes (centenas de GB pode levar 1 dia)
- **Tokenização:** Quebras sentença em bytes e aplicar as regras até obter os tokens (milhões de tokens por segundo em uma CPU)
- O maior custo envolvido com a tokenização é na matrix de embedding associada. Para cada token associamos um vetor. Então precisamos manter uma tabela de $|\mathcal{V}| \times d$ que pode ser grande ($\sim 50k \times 4096 \approx 800$ MB).

Positional Embeddings

Embedding sinusoidal:

$$E(x, i) = v_x + PE_i$$

$$PE_{i,2j} = \sin\left(\frac{i}{n^{2j/d}}\right) \quad PE_{i,2j+1} = \cos\left(\frac{i}{n^{2j/d}}\right)$$

Embedding absoluto:

$$E(x, i) = v_x + u_i$$

Embedding relativo:

(no cálculo da atenção)

$$e_{i,j} = \frac{x_i W^Q \left(x_j W^K + a_{i,j}^K \right)^T}{\sqrt{d_z}}$$

RoPE: Rotatory Position Embeddings

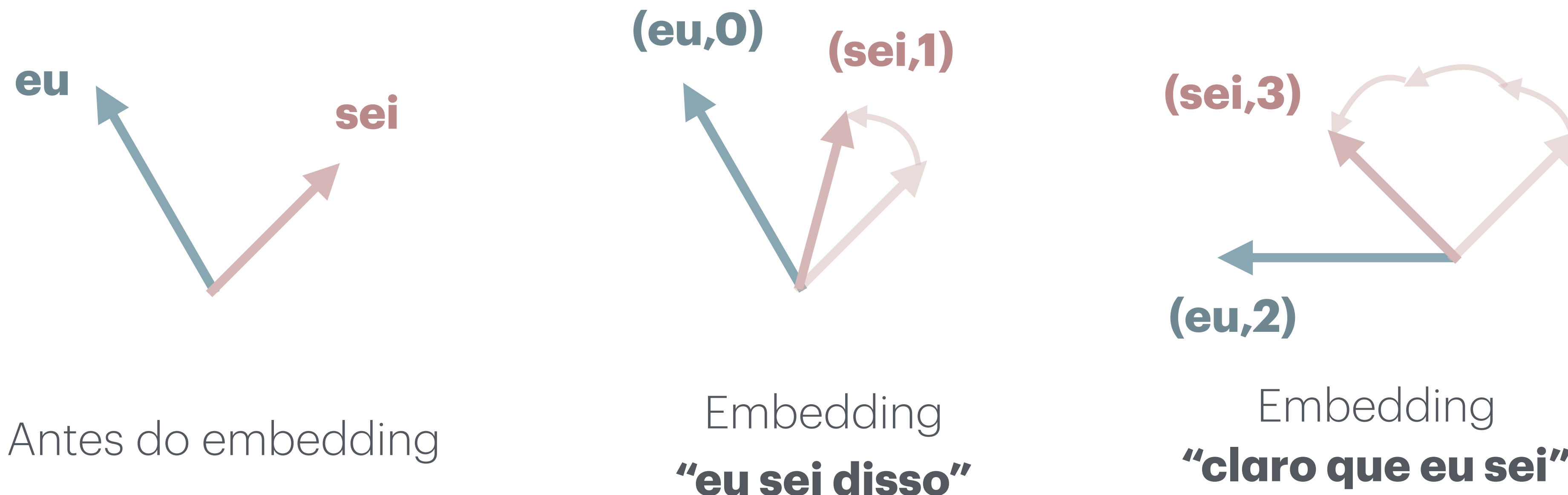
Gostaríamos de ter uma position embedding tal que a atenção entre os tokens seja dada em **função de suas posições relativas**,

$$\langle f(q, i), f(k, j) \rangle = g(q, k, i - j)$$

o que importa para comparar a importância de k para q deveria ser a posição relativa entre os tokens e não o número da posição em si.

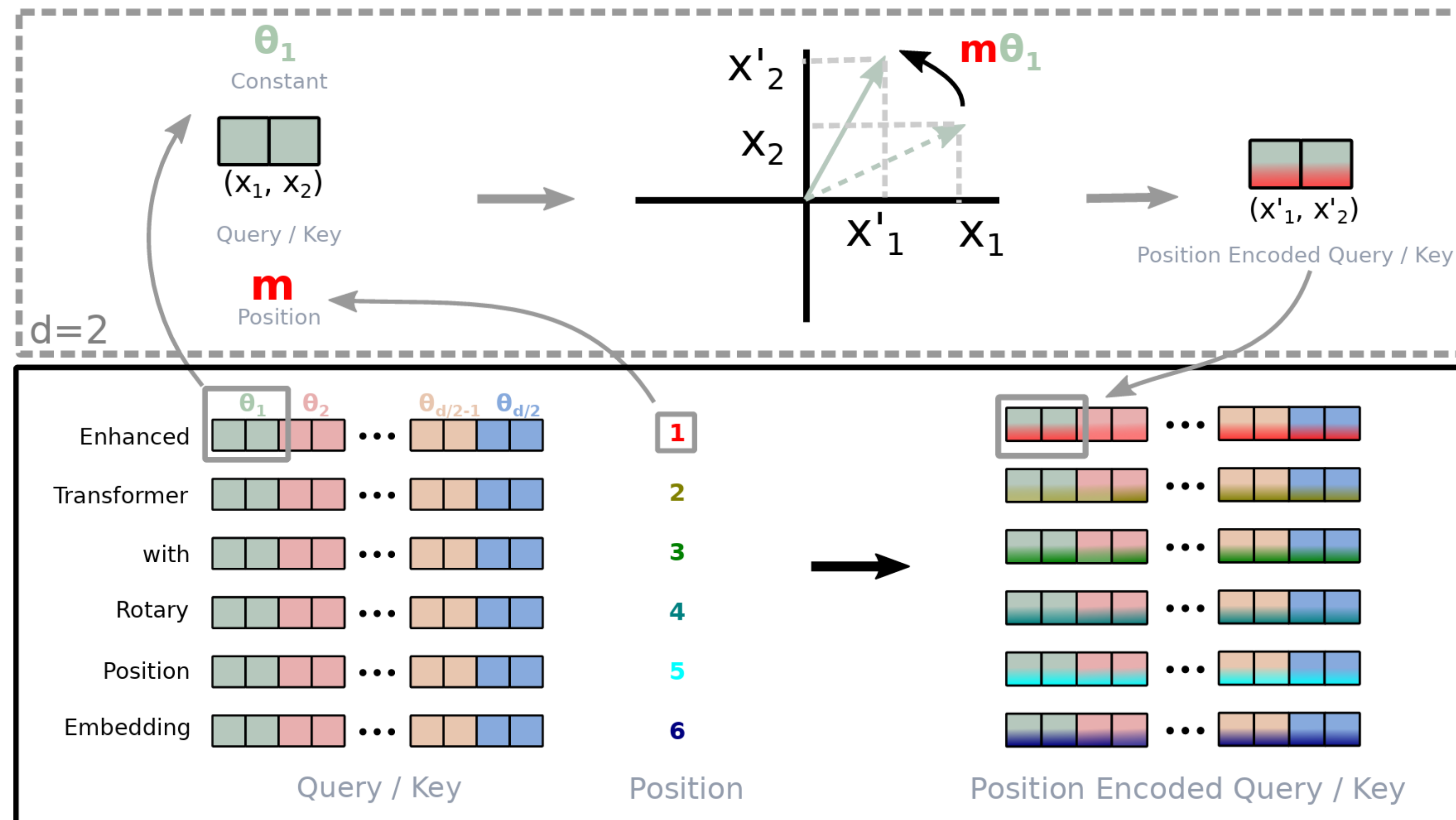
RoPE: Rotatory Position Embeddings

Em RoPE codificamos a posição usando rotações. Dado que $\langle x, y \rangle = \|x\| \|y\| \cos(\theta_{xy})$ é invariante por rotações, um mesmo shift na posição de dois vetores mantém a posição relativa.



RoPE: Rotatory Position Embeddings

Como fazer isso para vetores em alta dimensão?



RoPE: Rotatory Position Embeddings

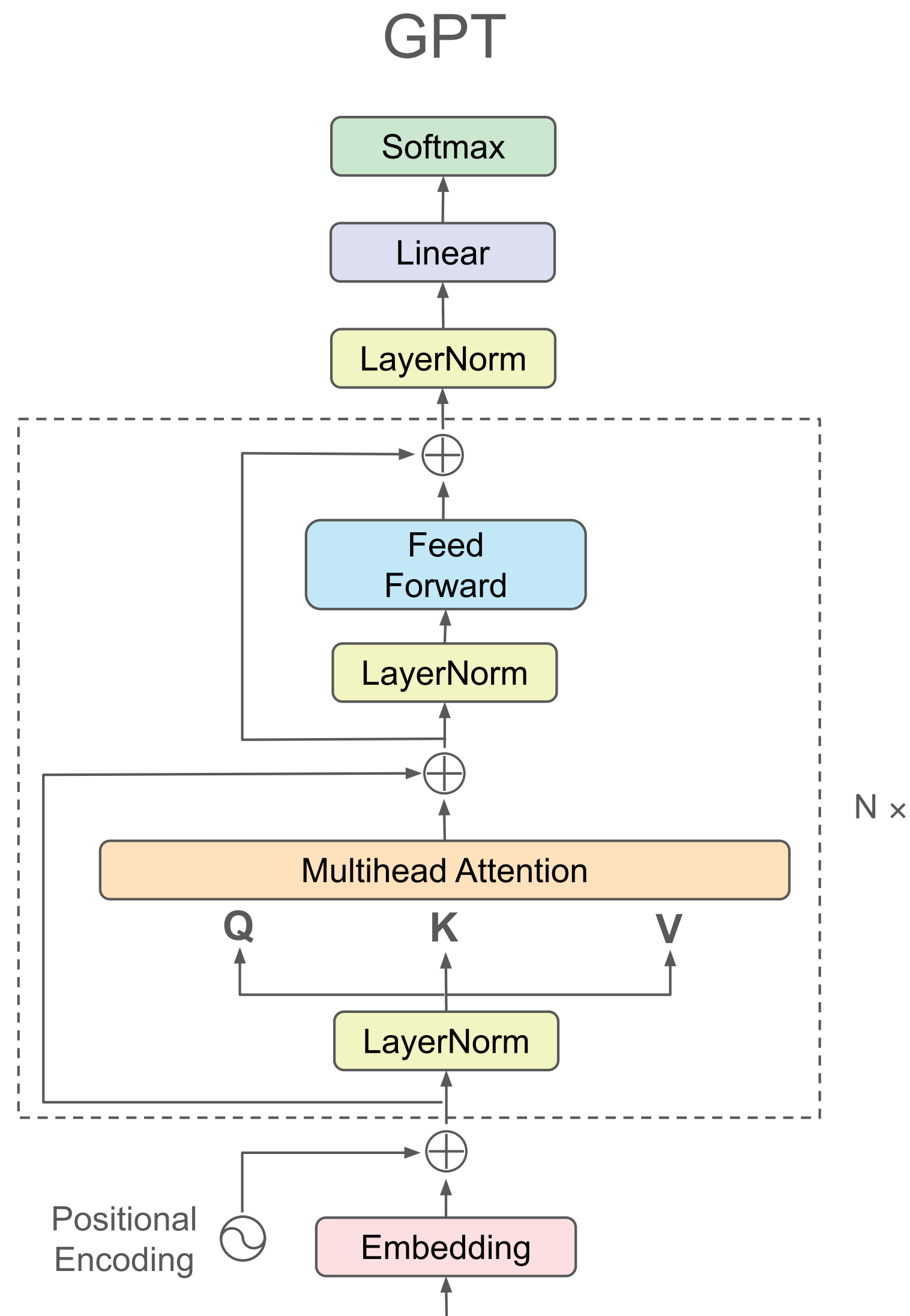
As operações podem ser escritas de forma matricial como

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m$$

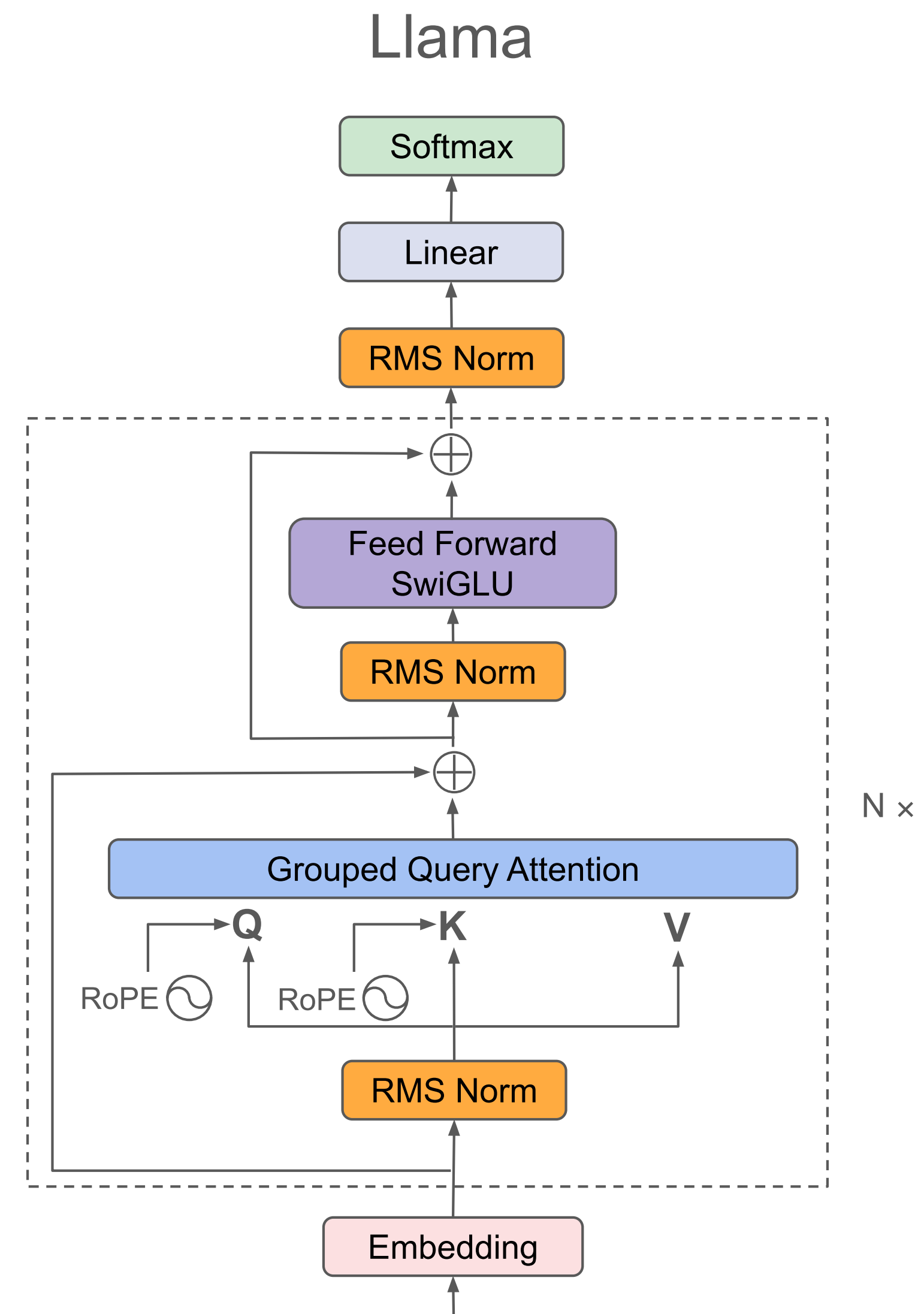
$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

$$\theta_i = 10000^{-2i/d}$$

Arquiteturas



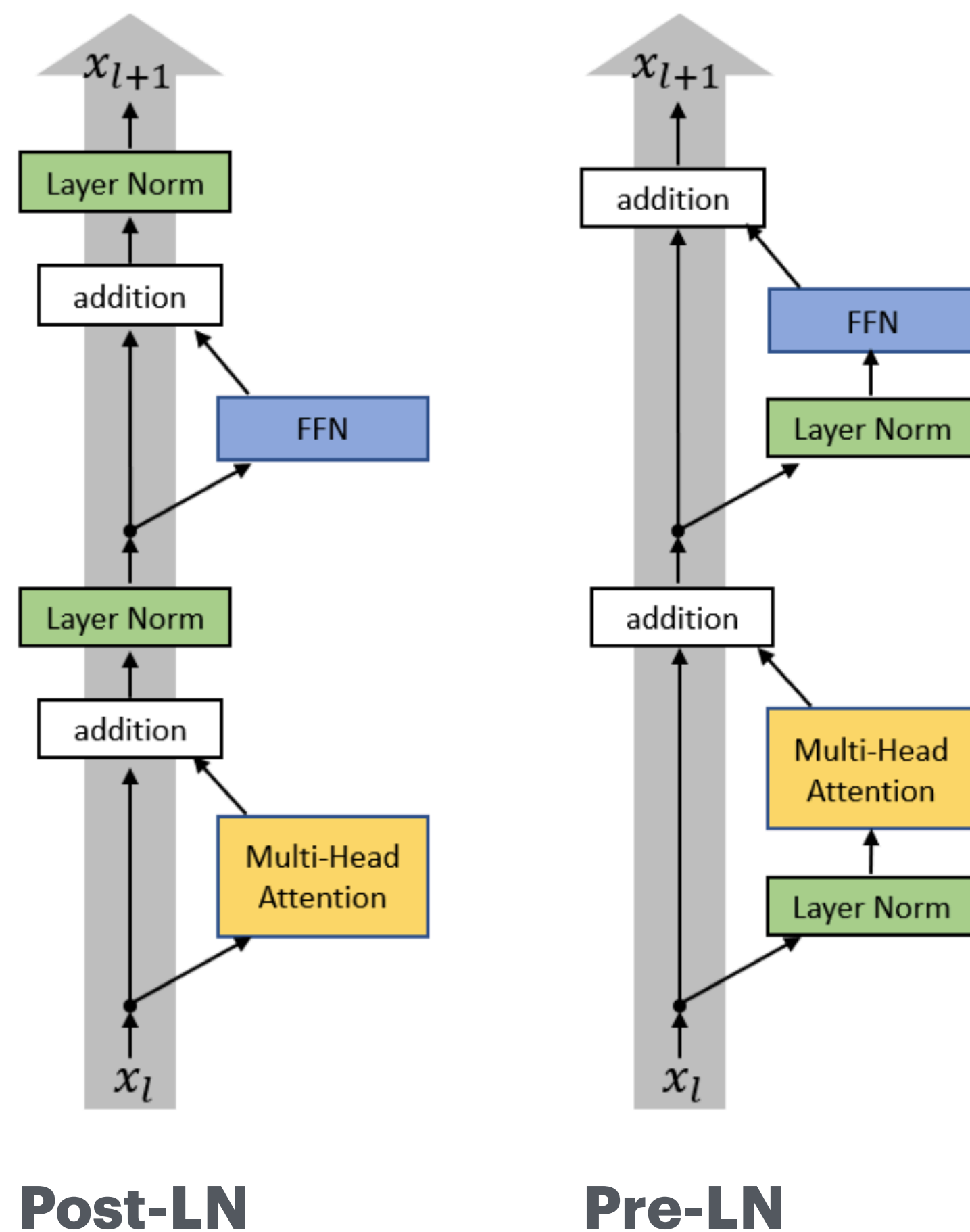
vs



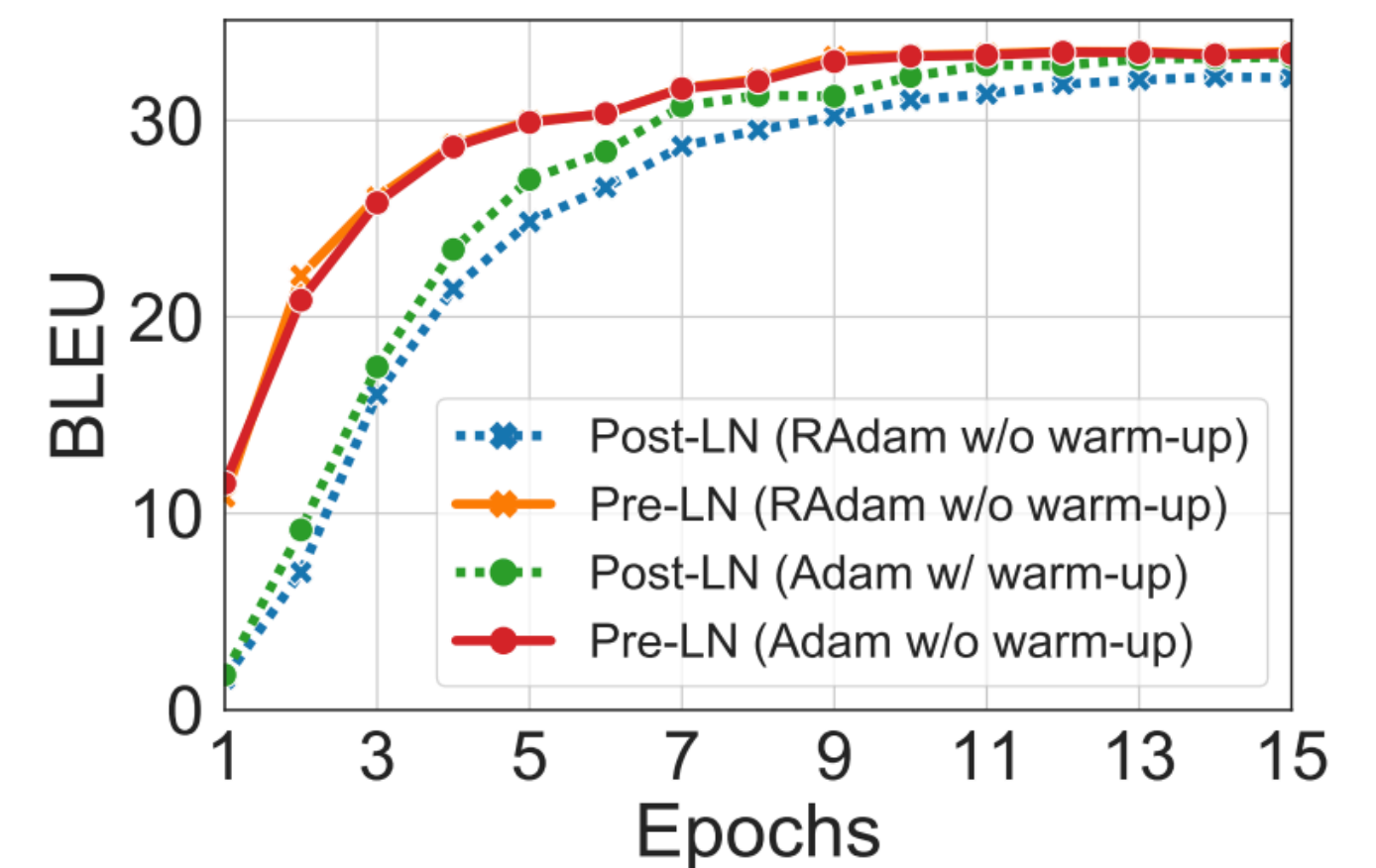
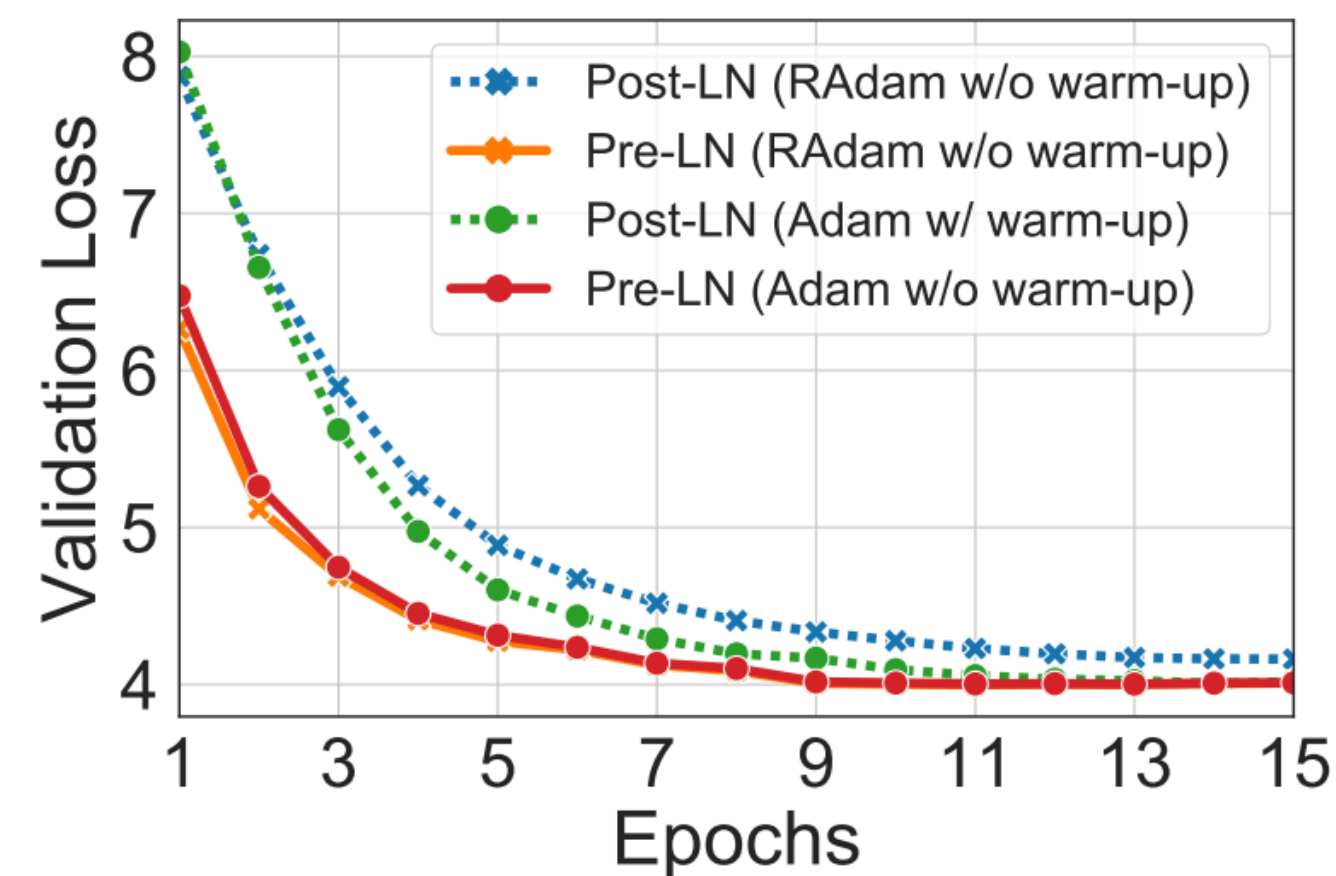
Arquiteturas

- Estratégias de Normalização
- Funções de Ativação
- Positional Embedding
- Hiperparâmetros - dimensões, tamanhos, proporções
- Truques de Estabilidade

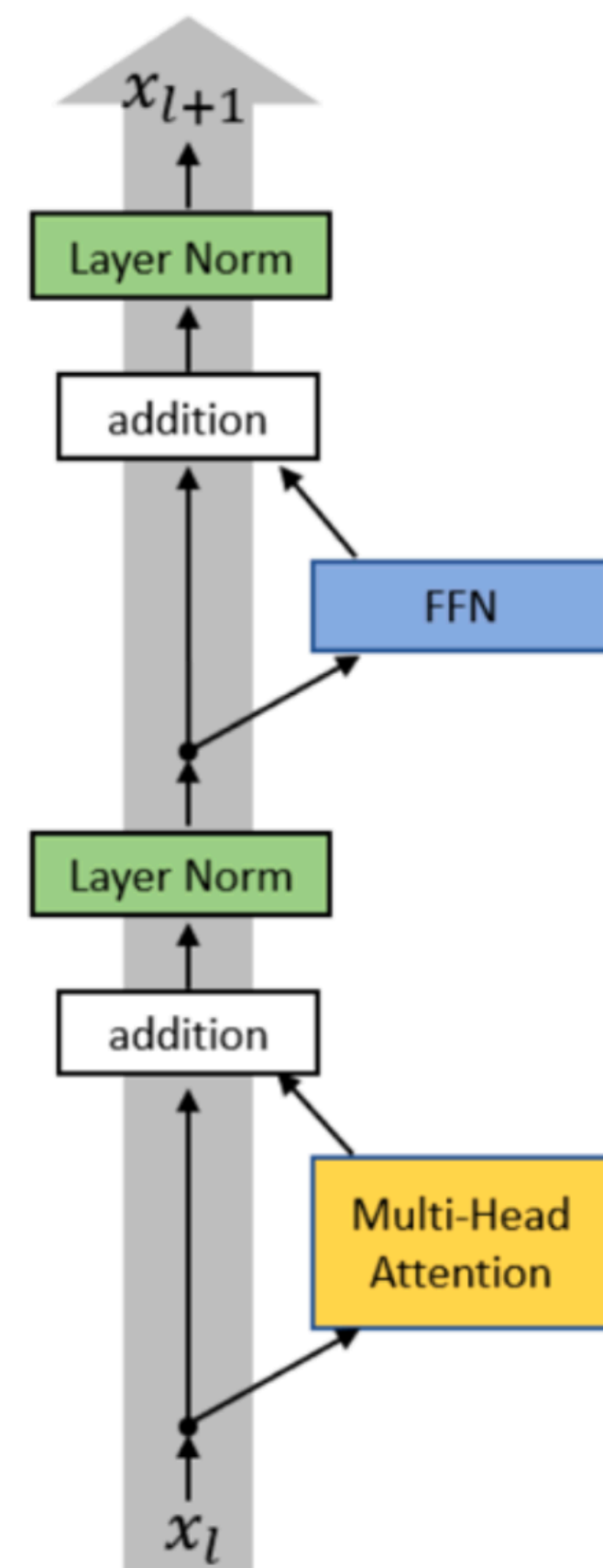
Pre-vs-Post Norm



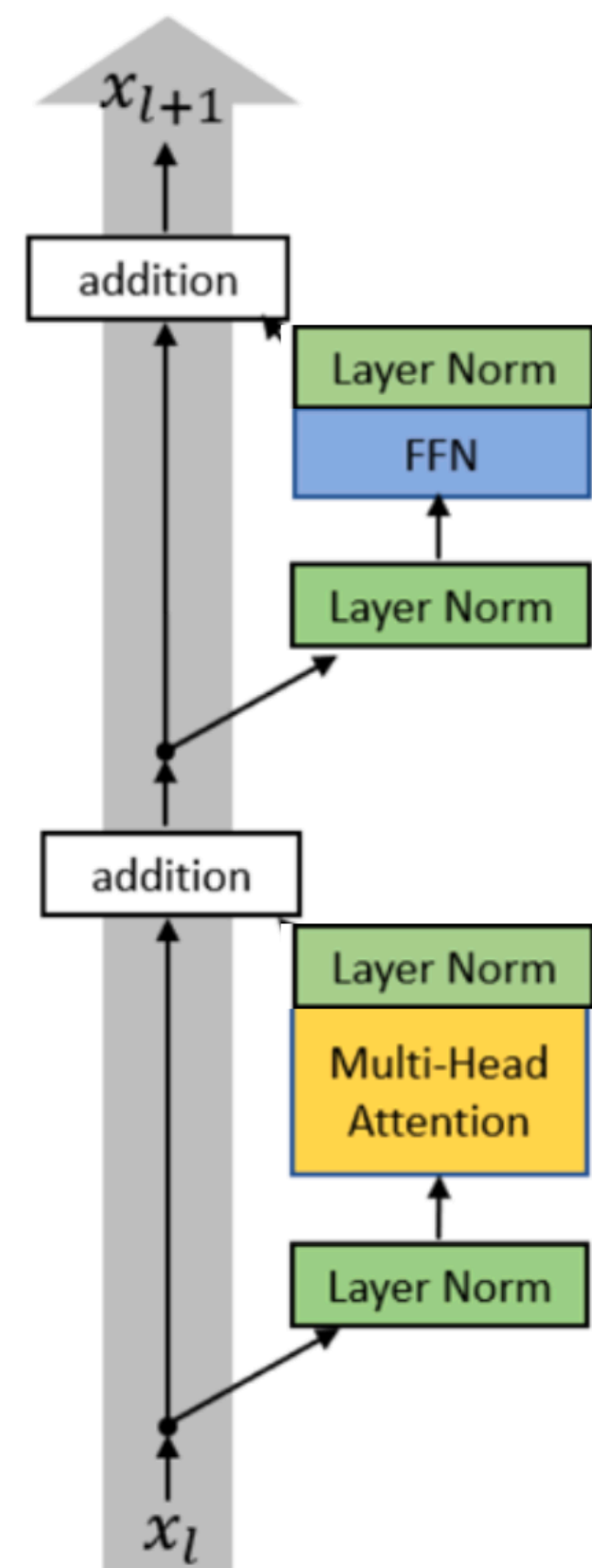
- Pre-LN mantém melhor a magnitude do gradiente no sinal residual
- Em geral, LLMs usam um esquema Pre-LN



Pre-vs-Post Norm



Post-LN



Pre-LN

- Alguns modelos recentes como Grok e Gemma 2 adicionam um Post-LN a mais, porém fora do caminho residual — “double” norm.
- Parece ajudar com estabilidade

LayerNorm vs RMSNorm

- **LayerNorm** normaliza a representação de cada token usando a média e variância do próprio vetor. Além de aplicar uma escala e bias aprendidos

$$x_{\text{out}}[k] = \frac{x_{\text{in}}[k] - \mathbb{E}[x_{\text{in}}]}{\sqrt{\text{Var}(x_{\text{in}})}} * \gamma + \beta$$

- **RMSNorm** normaliza sem a centralização ou escala da variância — ganho em eficiência

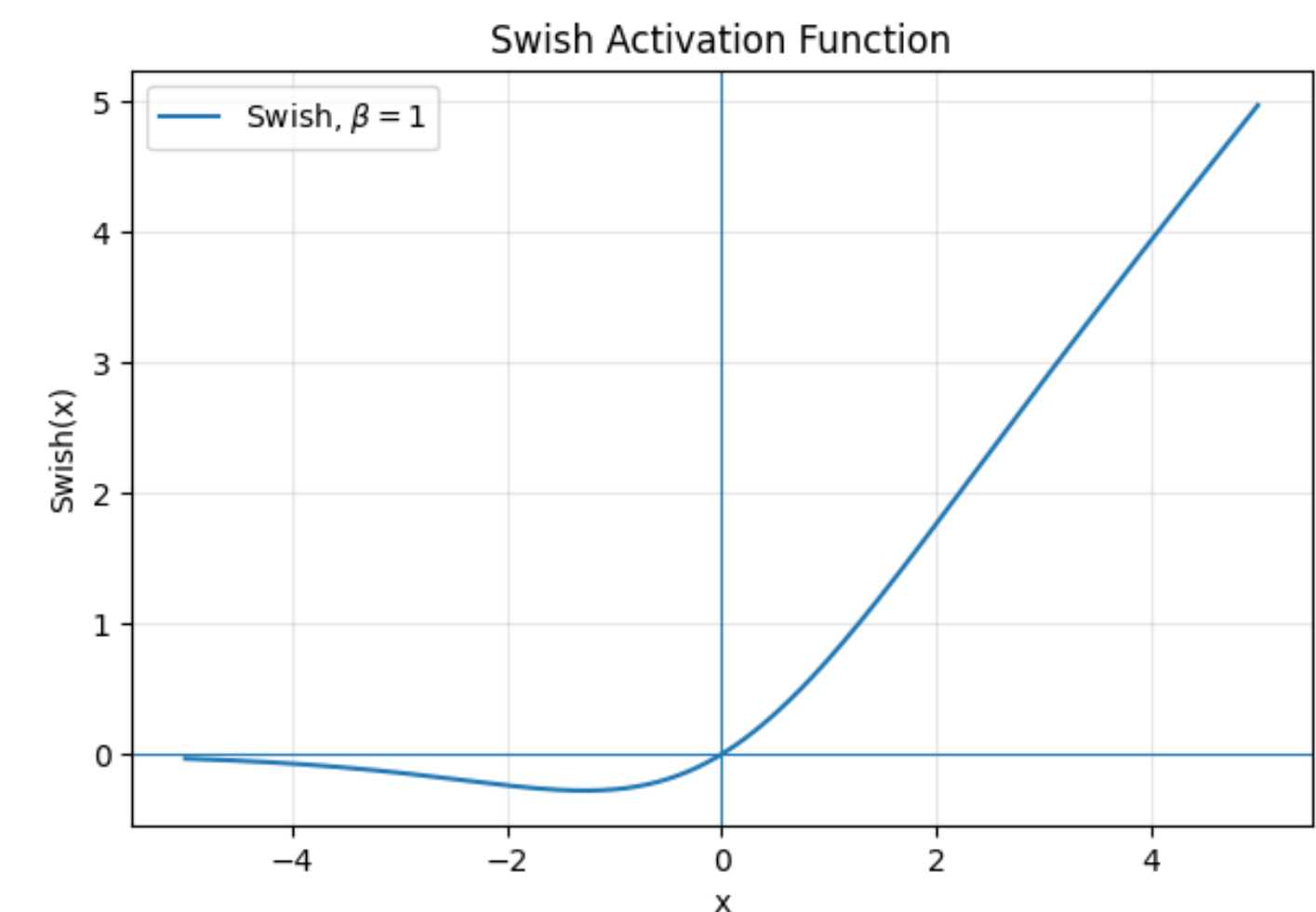
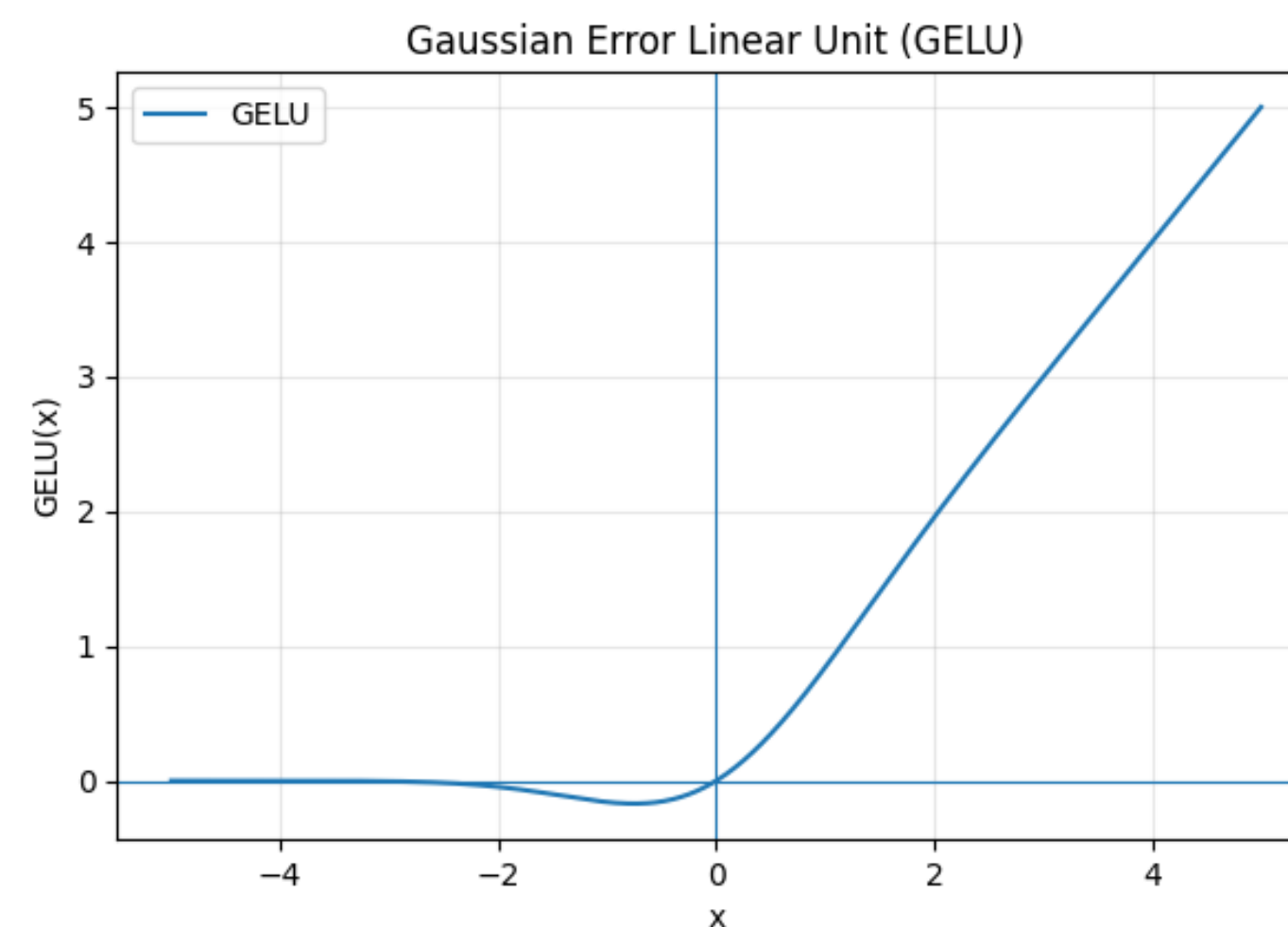
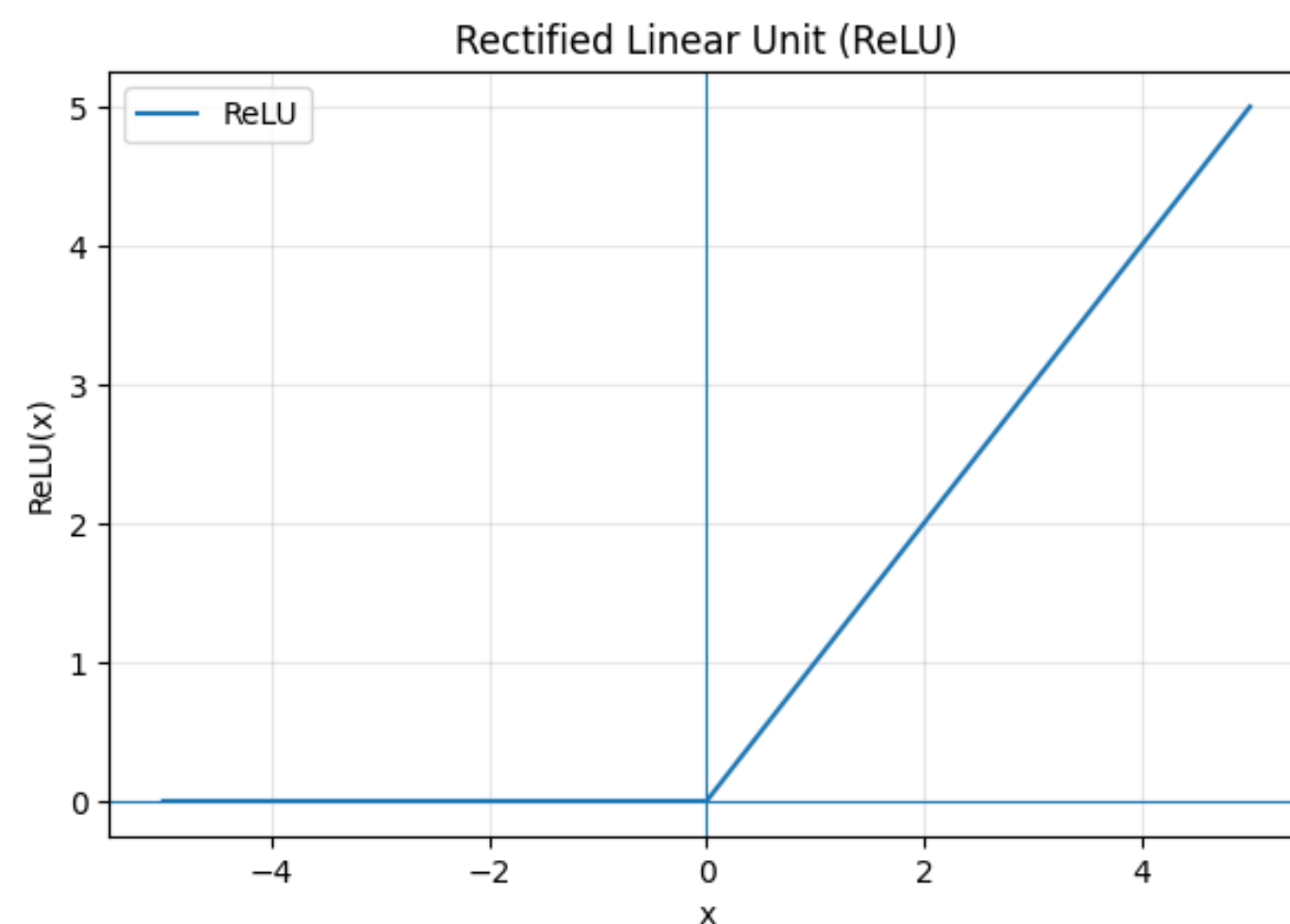
$$x_{\text{out}} = \frac{x_{\text{in}}}{\sqrt{\|x_{\text{in}}\|^2 + \epsilon}} * \gamma$$

Arquitetura Feed-Forward

A maioria dos transformers modernos **não possuem termo de viés**

$$\text{FFN}(x) = W_2 \max(0, W_1 x + b_1) + b_2 \quad \longrightarrow \quad \text{FFN}(x) = W_2 \sigma(W_1 x)$$

Funções de ativação: ReLU, GeLU, Swish, ELU, GLU, GeGLU, ReGLU, ...



Gated Linear Units (GLU)

Ativações com mecanismo de porta

Semelhante como em LSTMs, consideramos uma transformação linear que controla qual parte da informação é transmitida ($\otimes$ multiplica coordenada a coordenada)

$$\sigma(W_1x) \longrightarrow \sigma(W_1x) \otimes (Vx)$$

ReGLU:

$$\text{FFN}_{\text{ReGLU}}(x) = W_2 \max(0, W_1x) \otimes (Vx)$$

GeGLU:

$$\text{FFN}_{\text{GeGLU}}(x) = W_2 (\text{GeGLU}(W_1x) \otimes (Vx))$$

SwiGLU:

$$\text{FFN}_{\text{GeGLU}}(x) = W_2 (\text{Swish}(W_1x) \otimes (Vx))$$

Gated Linear Units (GLU)

Ativações com mecanismo de porta

	Score Average	CoLA MCC	SST-2 Acc
FFN _{ReLU}	83.80	51.32	94.04
FFN _{GELU}	83.86	53.48	94.04
FFN _{Swish}	83.60	49.79	93.69
FFN _{GLU}	84.20	49.16	94.27
FFN _{GEGLU}	84.12	53.65	93.92
FFN _{Bilinear}	83.79	51.02	94.38
FFN _{SwiGLU}	84.36	51.59	93.92
FFN _{ReGLU}	84.67	56.16	94.38
[Raffel et al., 2019]	83.28	53.84	92.68
ibid. stddev.	0.235	1.111	0.569

Hiperparâmetros

- O tamanho padrão para a camada escondida das redes Feed-forward é $d_{ff} = 4 d_{\text{model}}$, quando usamos GLU esse número é $d_{ff} = \frac{8}{3} d_{\text{model}}$.
- Para atenção com múltiplas cabeças o padrão é que $d_H = d_{\text{model}}/n_H$, porém algo um pouco maior também pode funcionar bem.

Pre-Treinamento

- **Preparação dos Dados:** deduplicação, segurança, privacidade, contaminação
- **Tokenização**
- **Escolha da Arquitetura**
- **Pré-Treino — Prever a próxima palavra**

$$\hat{\theta} = \arg \max_{\theta} \sum_{\mathbf{x} \in \mathcal{D}} \mathcal{L}_{\theta}(\mathbf{x}) \quad \mathcal{L}_{\theta}(\mathbf{x}) = \sum_{i=1}^k \log p_{\theta} \left(x_i \mid \mathbf{x}_{1:i-1} \right)$$

- **Otimização:** AdamW, Learning-rate scheduler, Dropout (?), ...

Avaliação

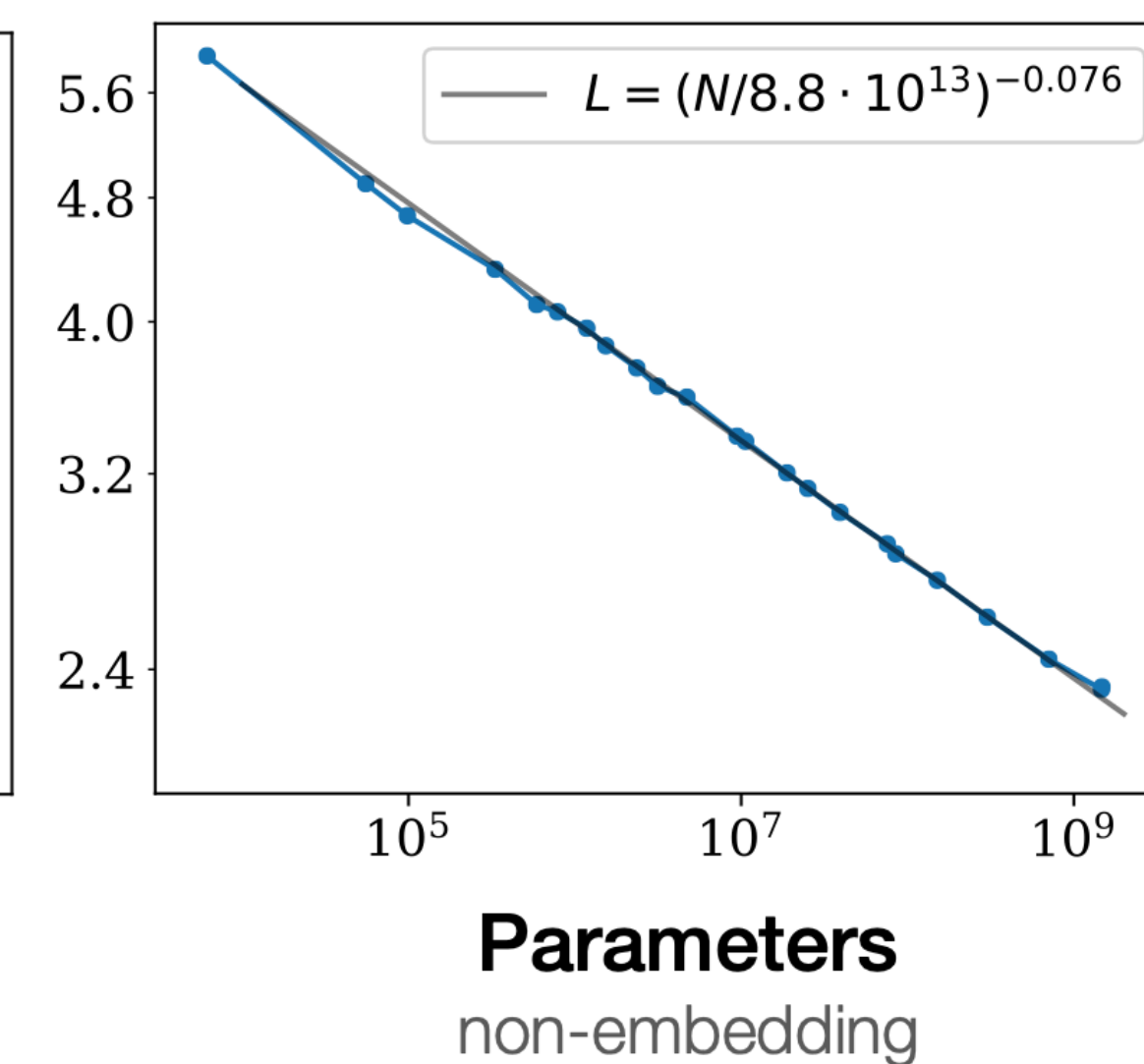
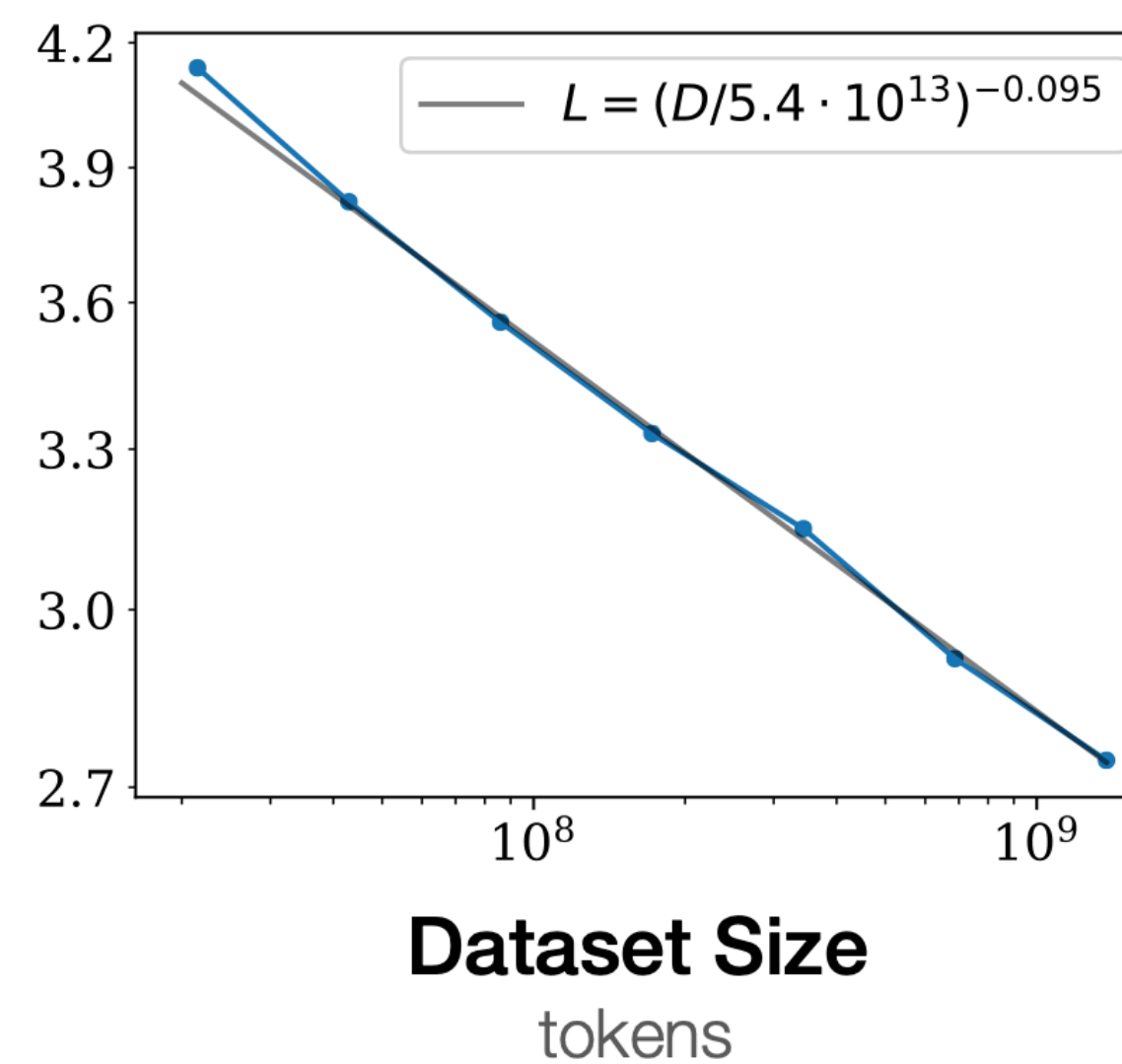
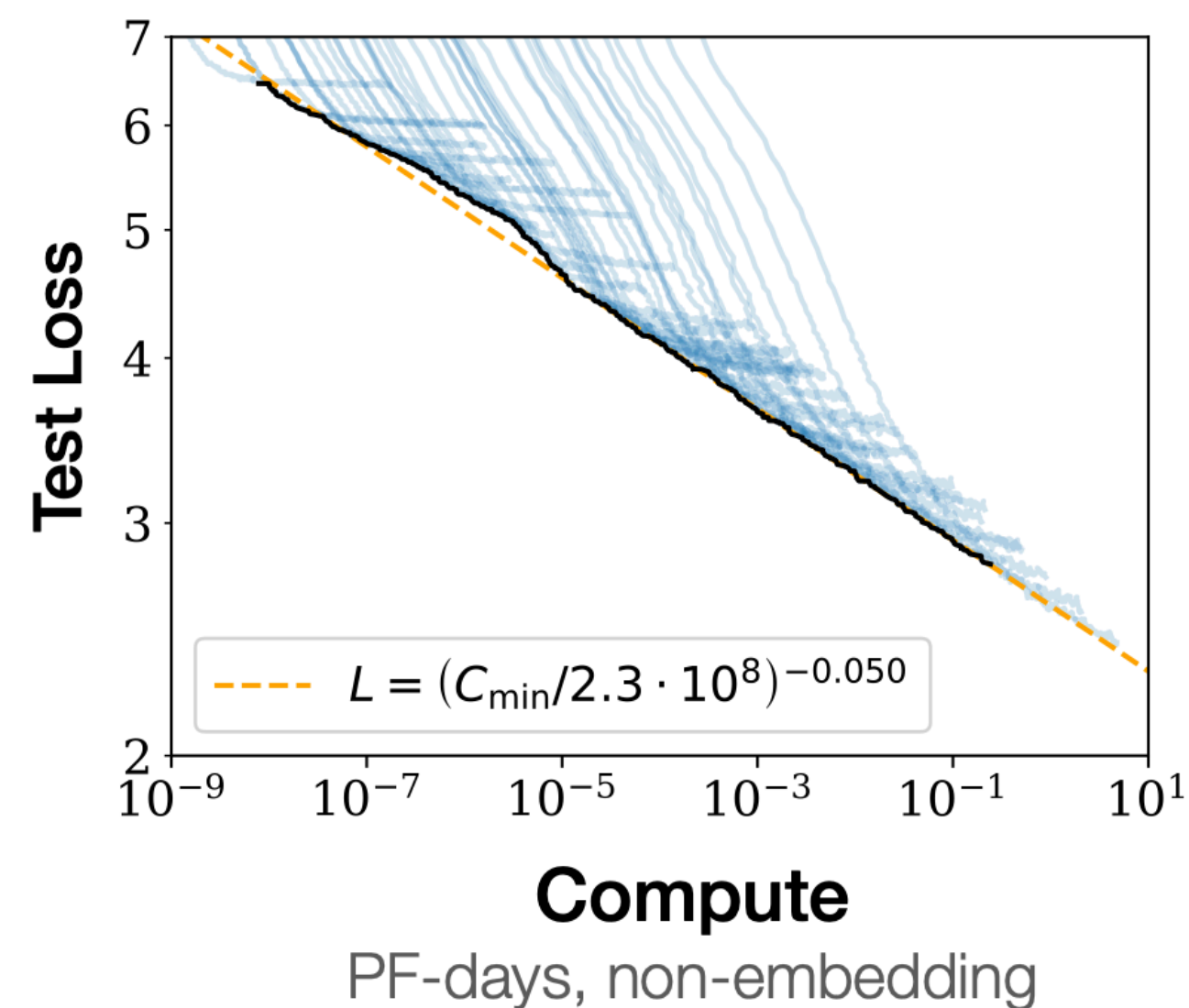
- **Perplexity** — média geométrica da verosimilhança — escala independente do tamanho da sequência

$$\text{PPL}_{\theta}(\mathbf{x}_{1:n}) = \left(\prod_{k=1}^n p \left(x_k \mid \mathbf{x}_{1:k-1} \right) \right)^{-\frac{1}{n}}$$

- **Zero-shot/Few-shot benchmarks**

Scaling Laws

- Como prever quais escolhas e fatores no pipeline de pre-treino afetam a performance do modelo?



Scaling Laws

Ex: Quanto tempo leva para treinar uma LLM com 70B parâmetros em 15T de tokens em 100 H100s?

- $N_{\text{flops}} = 6 \times 70\text{e}9 \times 15\text{e}12 = 6.3\text{e}24$
- $H100_{\text{flops_p_seg}} = 1979\text{e}12 / 2$
- $\text{mfu} = 0.5$
- $\text{flops_por_dia} = H100_{\text{flops_p_seg}} \times \text{mfu} \times 100 \times 60 \times 60 \times 10 = 4.27\text{e}21$
- $\text{Dias} = N_{\text{flops}} / \text{flops_por_dia} = 1475 \text{ dias } (\sim 4 \text{ anos})$

Scaling Laws

Ex: Suponha que lhe é dado acesso a duas mil H100s por 2 meses, e você precisa treinar uma LLM nesse tempo. O que você faz?

- Junta um grupo de pessoas para construir um pipeline de treino distribuído
- Coleta e prepara um conjunto grande de dados
- Escolhe um modelo grande

Scaling Laws

Ex: Suponha que lhe é dado acesso a duas mil H100s por 2 meses, e você precisa treinar uma LLM nesse tempo. O que você faz?

- Junta um grupo de pessoas para construir um pipeline de treino distribuído
- Coleta e prepara um conjunto grande de dados — **qual tamanho?**
- Escolhe um modelo grande — **quão grande? Qual modelo?**

Scaling Laws

Gostaríamos de entender a **dependência do erro de generalização** (a perda nos dados de teste) em função dessas escolhas, principalmente no **tamanho do dataset** (D) e o **número de parâmetros** do modelo (N). E de maneira similar com o custo computacional (C).

$$\mathcal{L}(D) = \frac{A_D}{D^{\alpha_D}}$$

$$\mathcal{L}(N) = \frac{A_N}{N^{\alpha_N}}$$

$$\mathcal{L}(C) = \frac{A_C}{C^{\alpha_C}}$$

$$\mathcal{L}(D, N) = \frac{A}{D^\alpha} + \frac{B}{N^\beta}$$

Power laws!

Scaling Laws

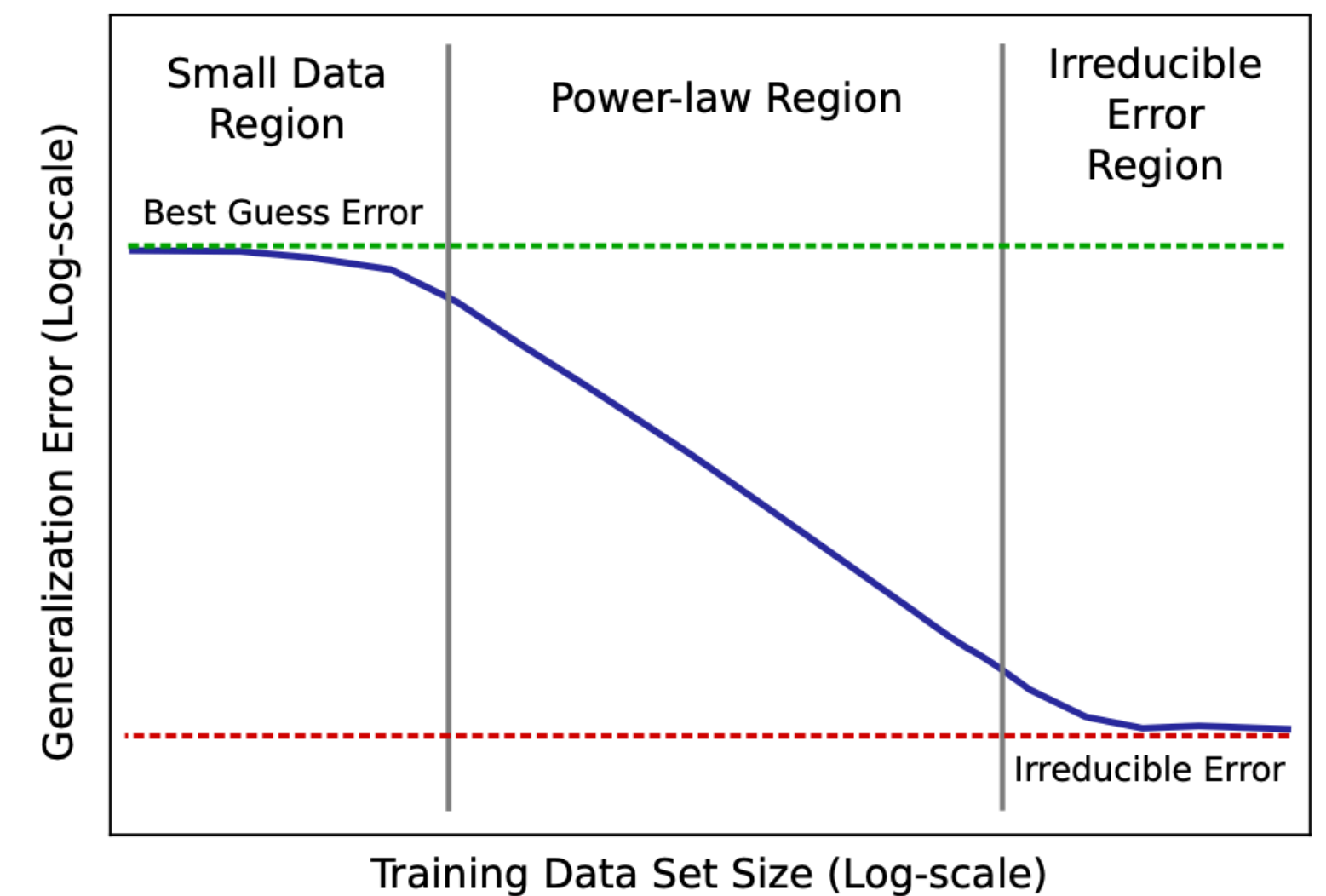
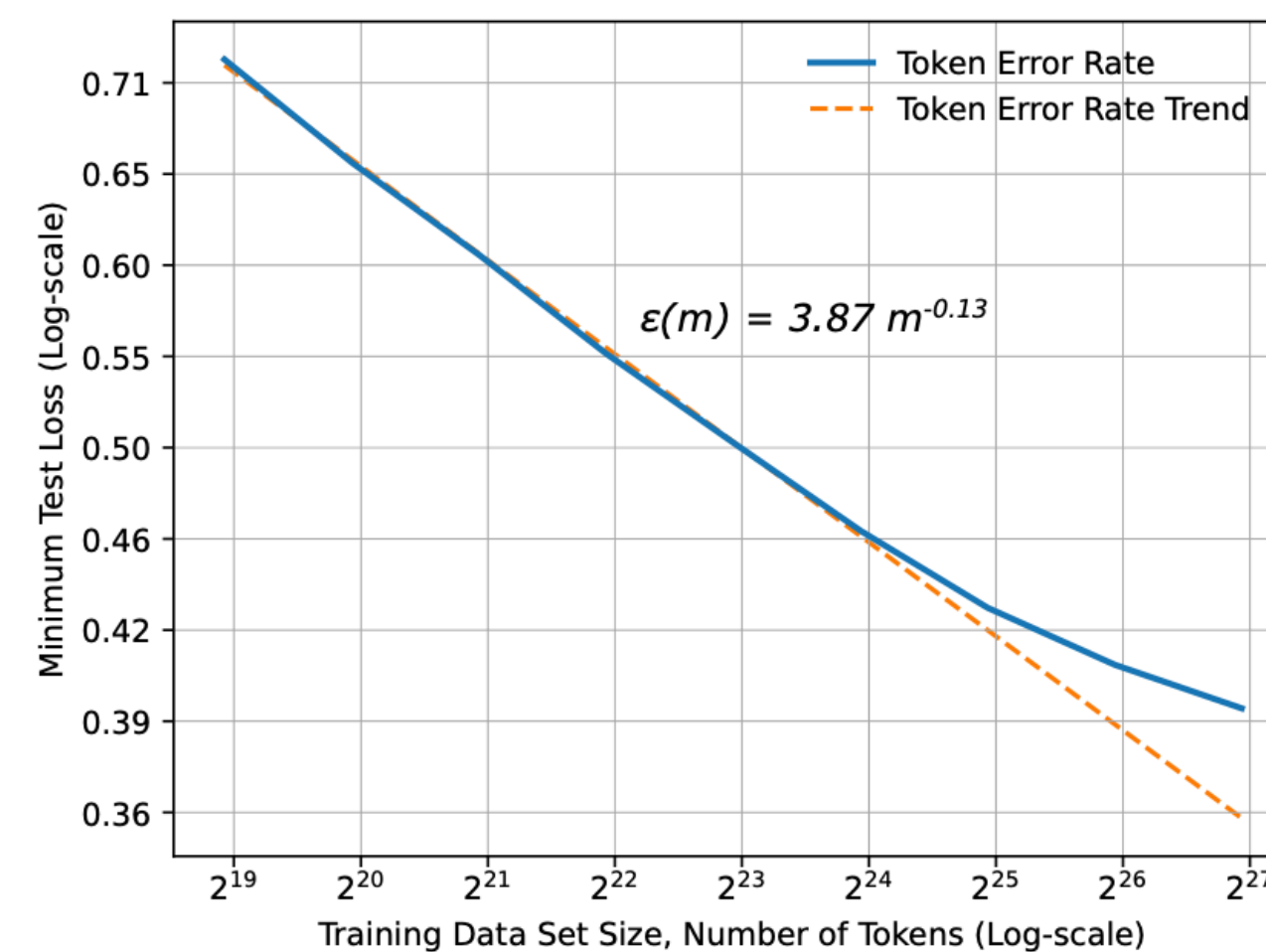
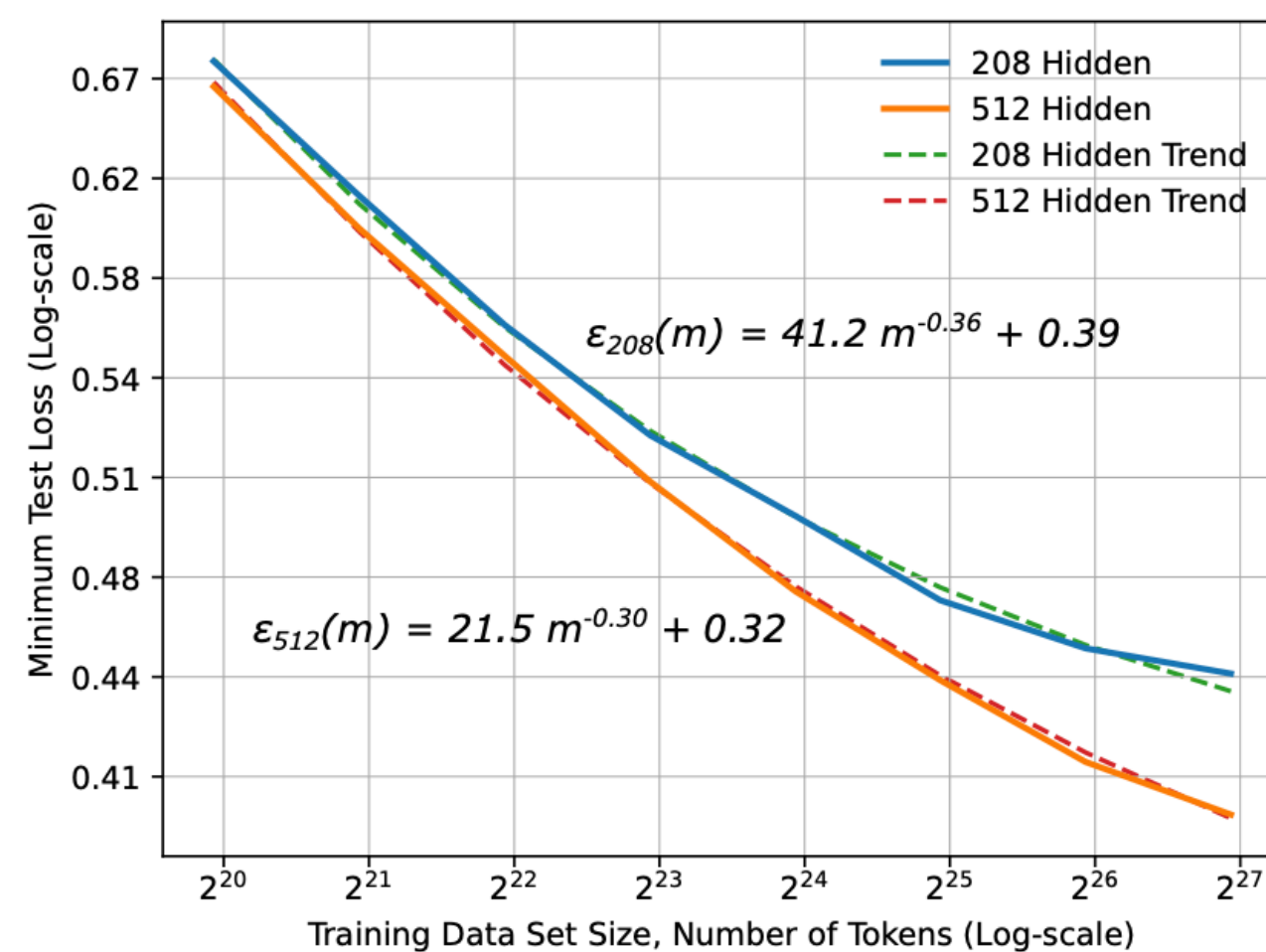
DEEP LEARNING SCALING IS PREDICTABLE, EMPIRICALLY

Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun,

Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, Yanqi Zhou

{joel,sharan,ardalaninewsha,gregdiamos,junheewoo,hassankianinejad,
patwarymostofa,yangyang62,zhouyanqi}@baidu.com

Baidu Research



Scaling Laws

Scaling Laws for Neural Language Models

Jared Kaplan *

Johns Hopkins University, OpenAI

jaredk@jhu.edu

Sam McCandlish*

OpenAI

sam@openai.com

Tom Henighan

OpenAI

henighan@openai.com

Tom B. Brown

OpenAI

tom@openai.com

Benjamin Chess

OpenAI

bchess@openai.com

Rewon Child

OpenAI

rewon@openai.com

Scott Gray

OpenAI

Alec Radford

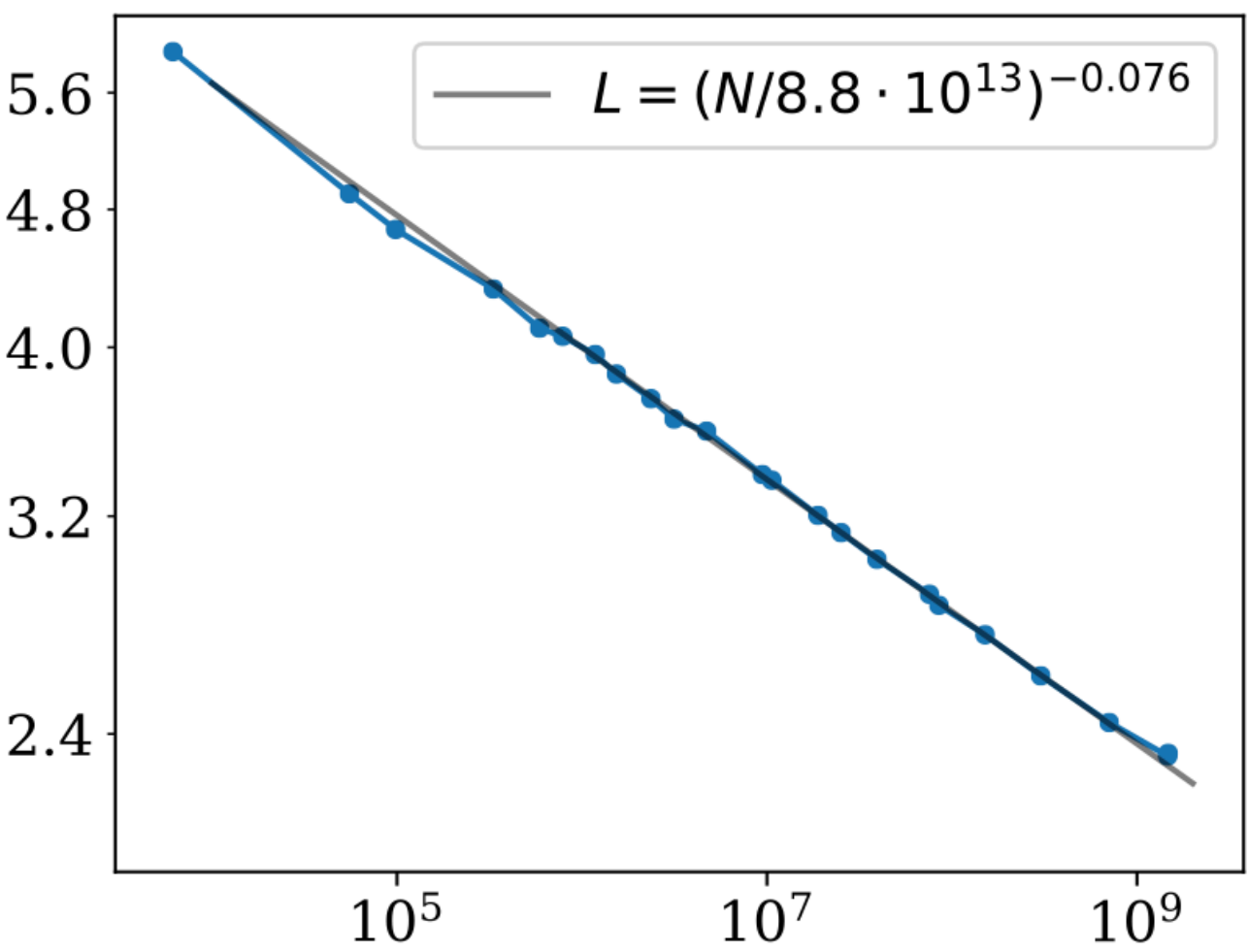
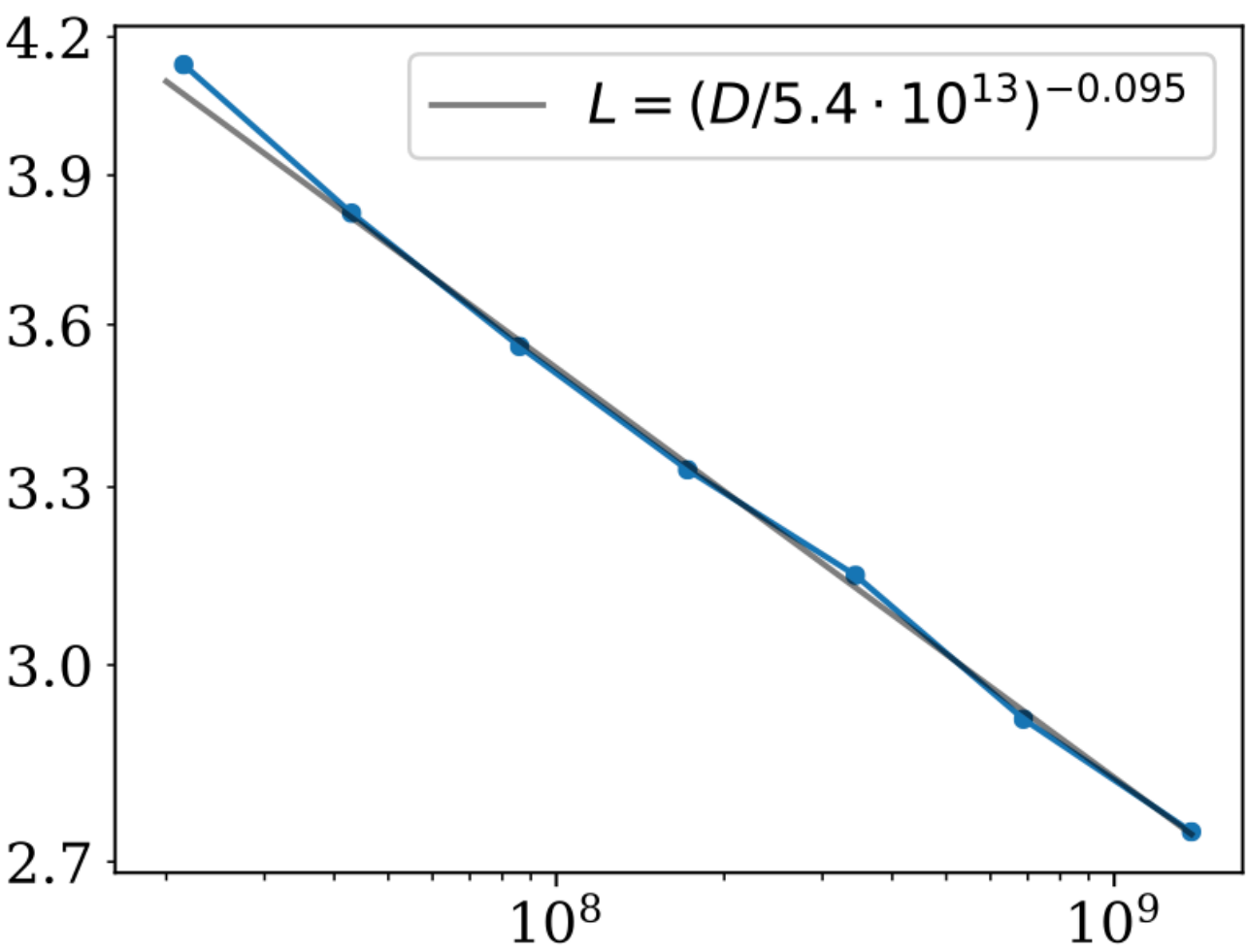
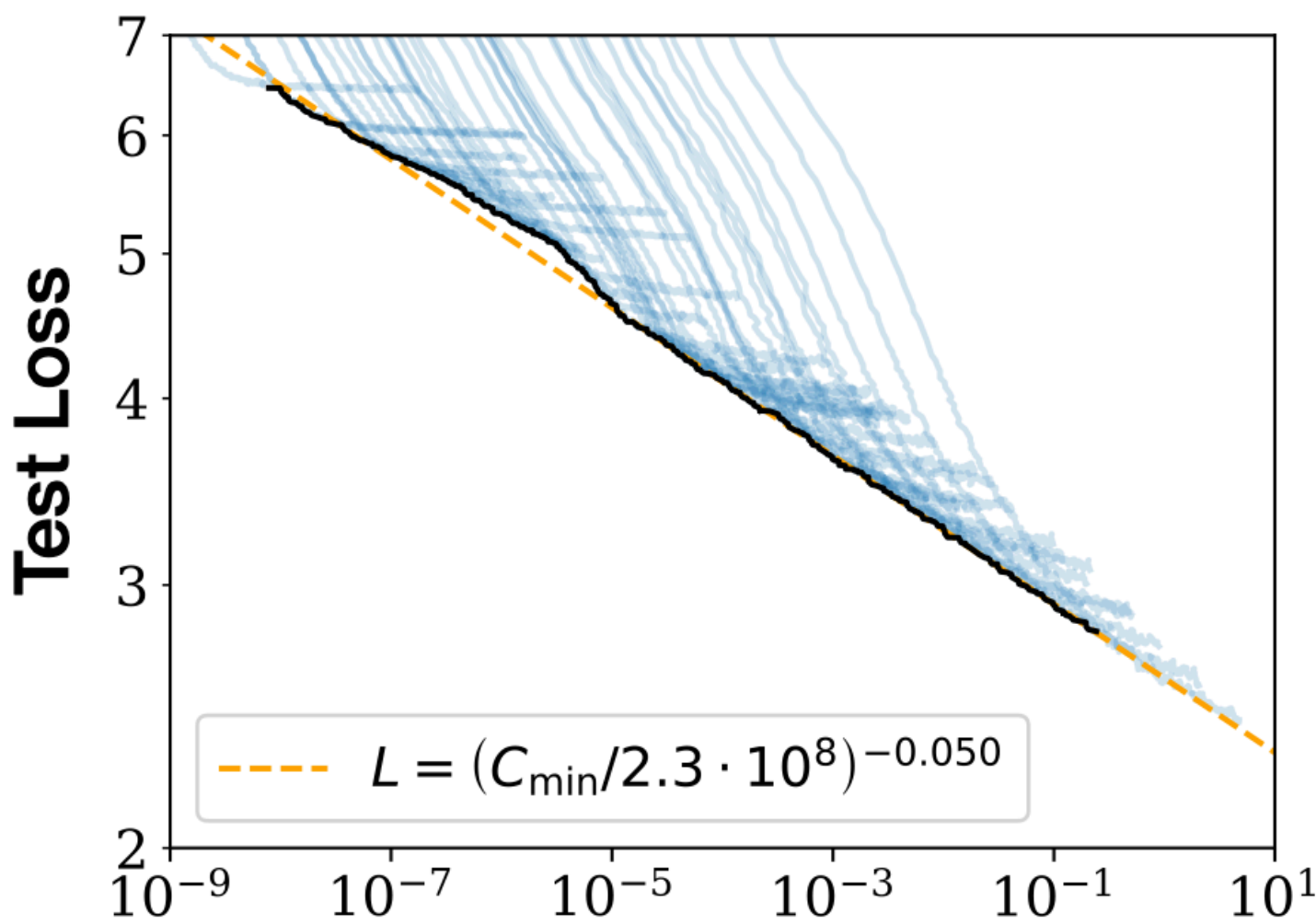
OpenAI

Jeffrey Wu

OpenAI

Dario Amodei

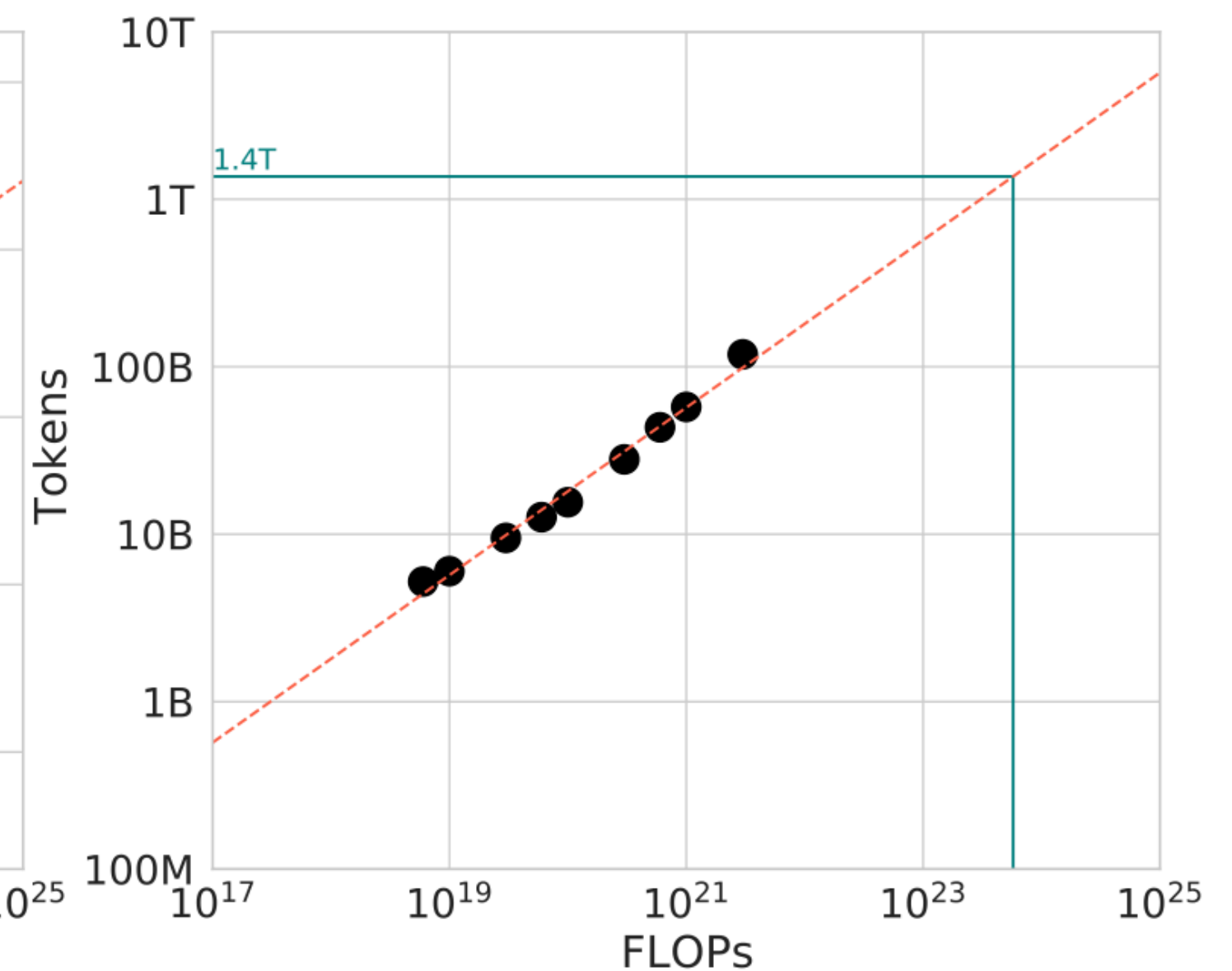
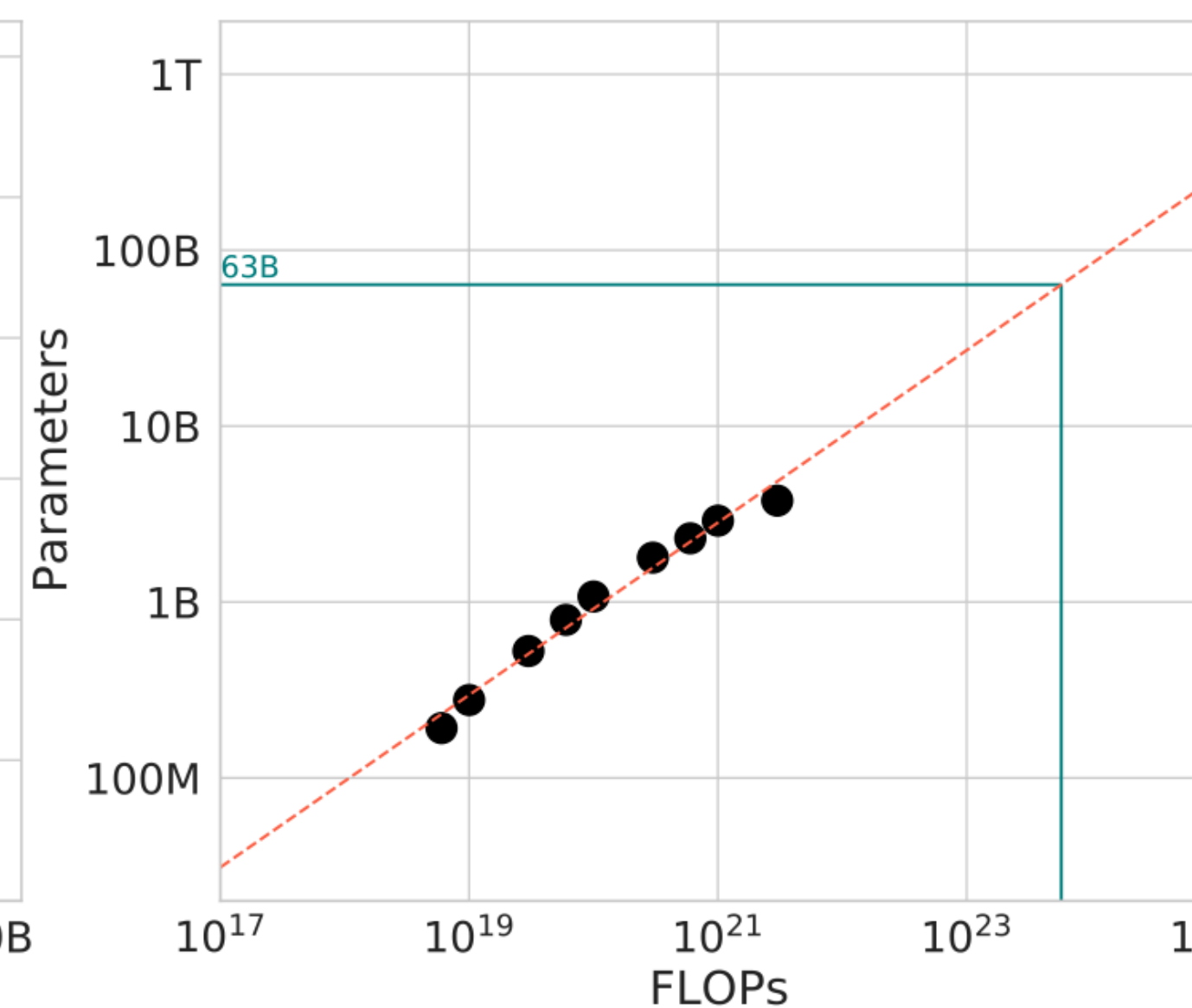
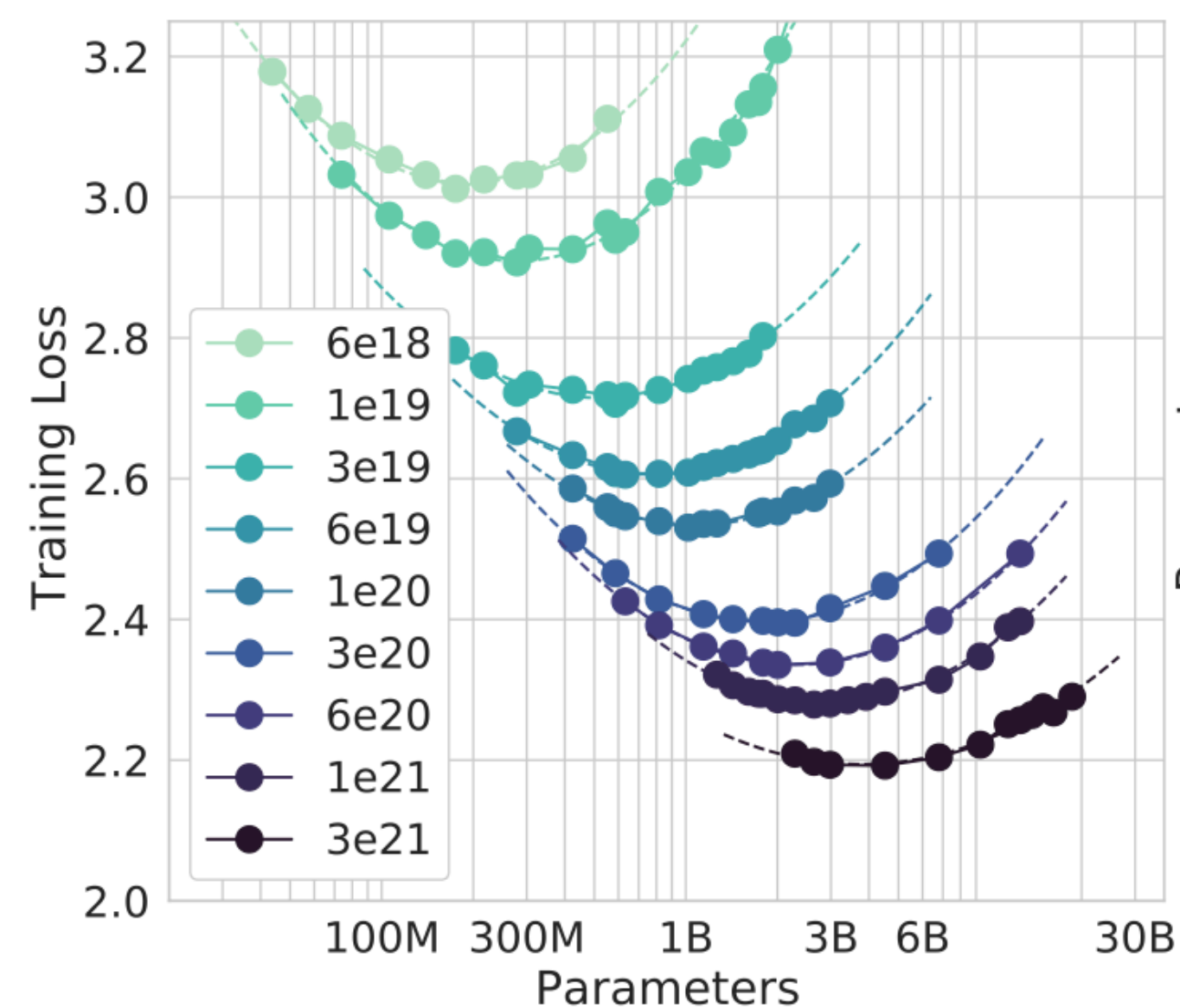
OpenAI



Scaling Laws

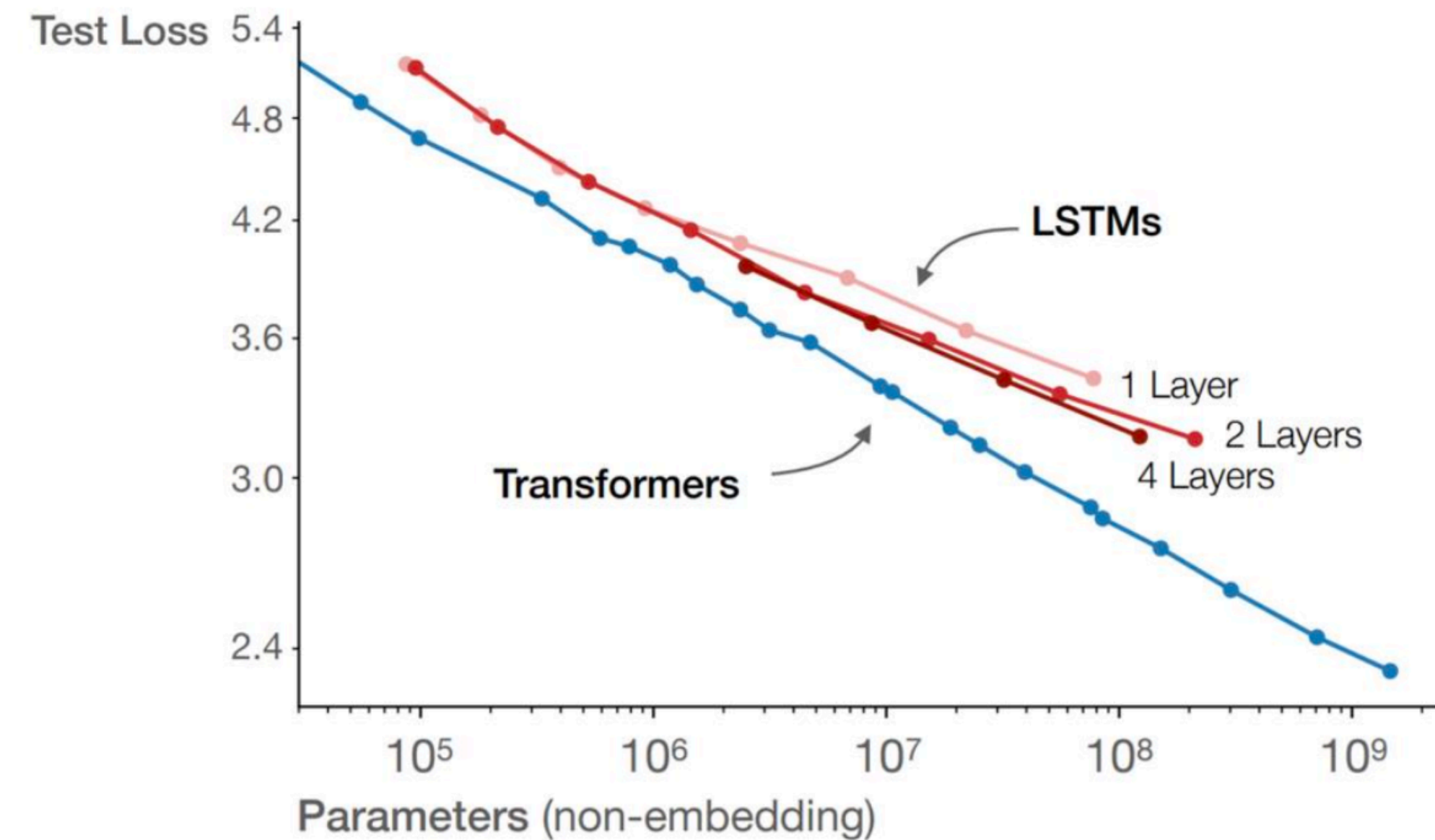
Curvas IsoFLOP

Fixado um orçamento de FLOPs (custo computacional), variamos o número de parâmetros e plotamos as perdas. Os mínimos de cada curva formam a power law

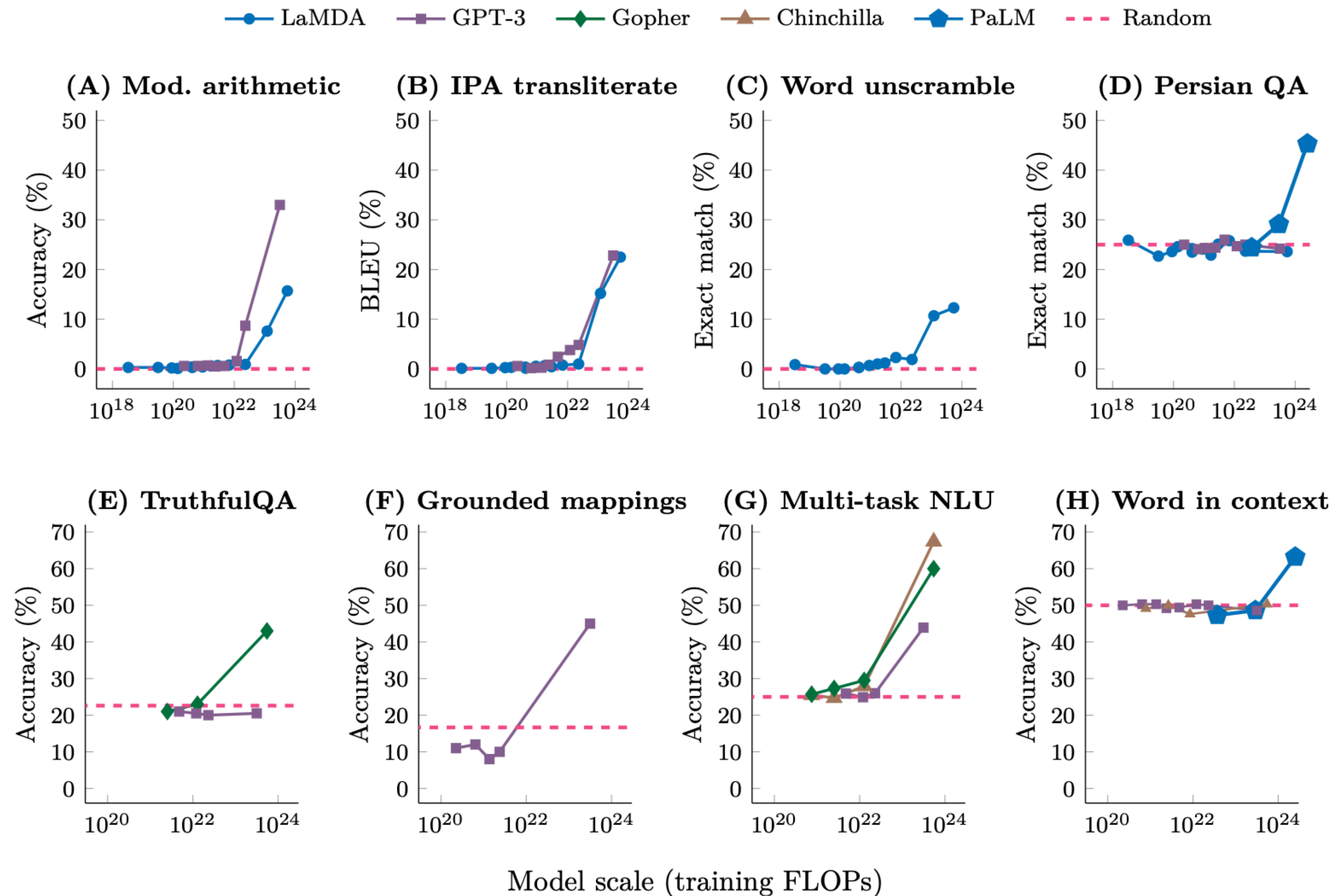


Scaling Laws

- Escolher mistura de dados ideal
- Efeito de repetir dados
- Qualidade dos dados
- Comparar Scaling laws entre arquiteturas
- Escolha do otimizador, numero de camadas, Batch size, learning rates
- ...



Habilidades Emergentes



Scaling Laws

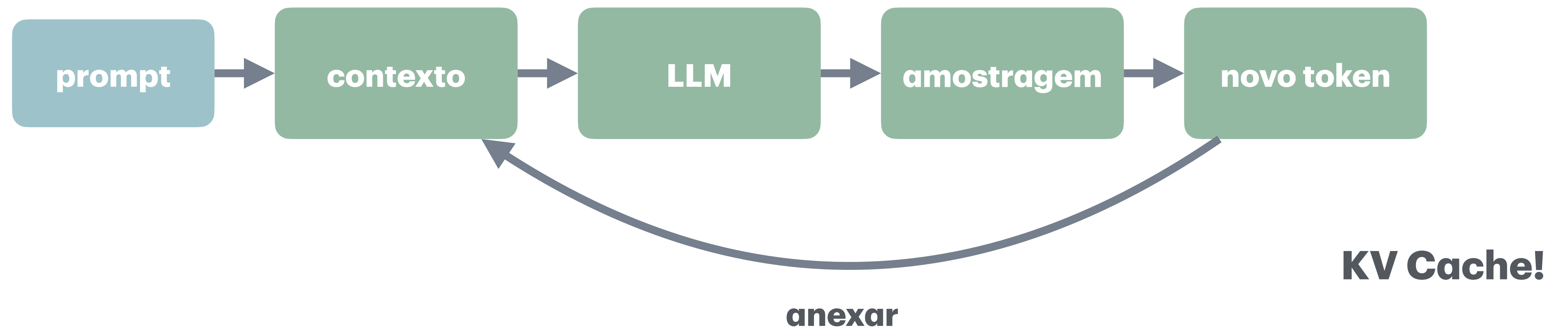
- A principal ideia de scaling laws é encontrar o tamanho ideal para um custo computacional fixo. Porém é importante lembrar que quanto maior o modelo, mais custoso fazer **inferência** nele
 - GPT3 – 2 tokens / param
 - Chinchilla – 20 tokens / param
 - Llama 2 70B – 29 tokens / param
 - Mistral 7B – 110 tokens / param
 - Llama 3 70B – 215 tokens / param

Se esperamos muito uso do modelo, pode valer a pena pagar um custo maior durante o pré-treino por um modelo menor.

Inferência

Geração autoregressiva:

- 1. Prefill:** O prompt é processado inteiro (em paralelo).
- 2. Decode:** Em cada passada, um token novo é gerado e anexado ao contexto (em sequência)



Inferência

Amostragem

- **Greedy decode:**

$$x_t = \arg \max_{v \in V} P(v \mid x_{<t}) = \arg \max_i z_i$$

z_i — logits

- **Temperatura:**

$$P(x_t = i \mid x_{<t}) = \frac{\exp(z_i/T)}{\sum_{j \in V} \exp(z_j/T)}$$

Inferência

Amostragem

- **Top-k:** Restringe aos k tokens mais prováveis — V_k

$$P'(x_t = i \mid x_{<t}) = \begin{cases} \frac{P(i \mid x_{<t})}{\sum_{j \in V_k} P(j \mid x_{<t})} & \text{se } i \in V_k \\ 0 & \text{se não} \end{cases}$$

- **Top-p:** Restringe ao conjunto de tokens $V_p = \{v_{(1)}, \dots, v_{(m)}\}$ tal que a probabilidade acumulada

atinge um limiar $m = \min \left\{ k : \sum_{i=1}^k p_{(i)} \geq p \right\}$

$$P'(x_t = i \mid x_{<t}) = \begin{cases} \frac{P(i \mid x_{<t})}{\sum_{j \in V_p} P(j \mid x_{<t})} & \text{se } i \in V_p \\ 0 & \text{se não} \end{cases}$$

Inferência

Amostragem

- **Beam search:** Idealmente gostaríamos de resolver o problema

$$y^* = \arg \max_y \prod_{t=1}^T P(y_t | y_{<t}, x)$$

Como buscar em todas as possíveis sequências é muito custoso, beam search faz uma busca mantendo os ***B*** caminhos mais prováveis, isso é feito através de um algoritmo por largura.

Inferência

Amostragem

$B = 2$

